五天学习指南

从零开始，为 11.6 会议任务做准备

这套资料只围绕本周任务：熟悉 RTL 编写、会议中提到的硬件开发工具、操作系统进程生命周期，并思考如何把高层安全目标转成具体技术目标。默认你没有数字电路、Scala 或操作系统基础。

你将接触一条完整链路：电路能够保存状态 → 用 RTL 描述电路 → 用 Chisel 生成 RTL → 操作系统管理运行中的程序 → 为 Agent 的启动、通信与停止寻找控制点。

资料怎么分

分册	主要解决的问题	建议用时
01 数字电路与 RTL	怎样读懂和编写最小硬件模块？	6 小时
02 Chisel 与工具链	Chisel、Verilog、仿真器各做什么？	3 小时
03 操作系统进程	程序怎样创建、运行、通信和退出？	5 小时
04 技术目标转化	怎样把课题愿景写成可验证的问题？	3.5 小时

另附 05《按操作系统运行代码》，按自己的系统查阅即可。它是五天内动手环节的操作手册，不额外增加一天。

建议每天约 3.5 小时，五天合计约 17.5 小时。“必学”先完成，“选读”按需查。新增语法补充 V1-V7 在 01 册、C1-C7 在 02 册，读代码时对照，不用一次背完；练习先作答再看答案。

一个需要保留的疑问

会议记录混用了 Trisim、Trisol 和 Chisel。结合后文明示的 Chisel 与 Chipyard，本资料按 Chisel 组织工具学习；这是一项有依据的解释，仍应向导师核实原词。Chisel 是硬件构造语言，不能直接理解成仿真器。[C1]

> 五天的目标是建立基础、做出小例子、提出初步技术目标。资料中的示意架构、候选指标与实验表格都是教学内容。

资料编写日期：2026-11-07。研究背景依据本地题目图片与 11.6 会议记录；技术参考优先采用项目官方文档、Linux 手册和 Python 文档。

五天时间表

每天完成一个小闭环

天数	时间安排	当天留下的成果
第 1 天	90 分钟读 01 基础与组合电路，查 V1-V3；60 分钟真值表、二进制；60 分钟读寄存器、计数器	选择器真值表；计数器逐拍表
第 2 天	60 分钟读状态机、测试并查 V4-V5；90 分钟运行 RTL；60 分钟订正，按需读 V6-V7	PASS 输出或手工推演；解释一段 RTL
第 3 天	90 分钟读 02 并查 C1-C3；60 分钟生成 RTL；60 分钟对照语法，C4-C7 按需查阅	工具链图；两种语言对照笔记
第 4 天	100 分钟读 03 的进程与内存；70 分钟运行 Python 示例；40 分钟画生命周期和通信图	进程生命周期图；父子进程运行记录
第 5 天	90 分钟读 04；60 分钟填写技术目标模板；60 分钟完成综合自测、口头汇报	一页技术目标草案；三分钟汇报提纲

每天在中间留 2 次 10 分钟休息，额外计入日程。若有一节超过预计时间，先完成必学，再回头看选读。

遇到不懂的词怎么办

第一遍只问“它是什么、在整个链路的哪里、输入输出是什么”。例如先把 MMU 理解成“受操作系统配置的地址翻译与访问权限硬件”，暂时跳过页表格式。第三次遇见同一词，再返回分册内的定义复习。

每天都做这三个动作

- 用自己的话复述一个概念，禁止只抄定义。
- 对一个示例预测输出，再实际运行或手工核对。
- 写下一个与课题有关的问题，例如“停止主进程后，子进程一定停止吗？”

如果当天只有 2 小时，优先读概念、完成逐拍表和自测；把安装和扩展阅读安排到后面。五天覆盖基础需要持续投入，快慢以自测结果为准。

先认识你的实验环境

文件与命令，从最少的开始

所有 PDF 位于“学习资料”，共用源代码位于“配套示例”。该目录另设 Windows、macOS、Ubuntu-WSL 三个入口，各有独立说明。先读 05 中自己的系统部分，再进入示例目录。

三个系统，三条入口

系统	在哪里输入命令	先运行什么
Windows	PowerShell	py -3 run_examples.py python
macOS	终端 Terminal	bash macOS/run_examples.sh python
Ubuntu / WSL	Ubuntu 终端	bash Ubuntu-WSL/run_examples.sh python

终端接收文字命令，当前目录决定相对文件名指向哪里。必须先进入“配套示例”，才能直接使用上表中的相对路径。

Windows 的作者路径示例：

```powershell
Set-Location -LiteralPath 'D:\Projects\毕设\学习资料\配套示例'
py -3 run_examples.py python
```

macOS 若把仓库放在下载目录：

```bash
cd "$HOME/Downloads/Graduation-Project/学习资料/配套示例"
bash macOS/run_examples.sh python
```

路径按自己的位置修改，也可输入 `cd ` 后把文件夹拖进 Mac 终端。WSL 中 D 盘通常写为 `/mnt/d/`；它是 Windows 的 Linux 环境，不能把 apt 命令直接放进 PowerShell。

三套入口共用教学源代码。将末尾的 python 改成 rtl 或 chisel，可分别运行仿真或生成 RTL；请先按 05 安装对应工具。Python 例子只用标准库。

哪些内容属于 Linux

进程创建、等待、通信的基础可在 Mac 学习；/proc、cgroup 与 Linux namespace 等专项机制需要 Linux 环境，不能在 macOS 原生终端原样执行。

安装卡住时的处理

先分清“找不到命令”“目录不对”与“下载失败”。安装最多占用本次学习 30 分钟，之后先做手算。macOS 步骤已核对官方说明，当前制作环境未在实体 Mac 实测；详细记录见配套说明。

五天结束时，怎样判断学到了

最低完成清单

- 能把 1010 换算成十进制，并解释 4 位无符号计数器为何会从 15 回到 0。
- 能区分组合电路与时序电路，指出计数器里的状态、时钟和复位。
- 能解释 RTL、Chisel、仿真器、FPGA 的关系，画出工具链。
- 能画出进程创建、就绪、运行、阻塞、退出、回收的关系。
- 能说明 fork、exec、wait 的区别，理解这些是 Linux / Unix 语境。
- 能说明主进程退出与整个 Agent 停止之间有哪些待核实的环节。
- 能给三个课题目标各写出保护对象、假设、控制点、指标和一项实验。

学习记录模板

```text
日期 / 实际用时：
今天读了：
我能解释的三个词：
我预测的输出 / 实际输出：
我做错的一题及原因：
与课题有关的一个问题：
明天先解决什么：
```

三分钟组会汇报框架

第 1 分钟：解释一个 Agent 可能由控制程序、推理服务、工具进程、文件与网络依赖组成，说明你暂定的管理边界。

第 2 分钟：展示小模块的逐拍行为，以及进程从创建到回收的图。用一个例子解释“控制一个状态位”与“让系统中的行为停止”之间还差哪些接口。

第 3 分钟：提出三条候选技术目标，说明每条怎么测、依赖什么信任假设，并列出需导师明确的两三个问题。

阅读来源

[C1] Chisel 官方介绍：https://www.chisel-lang.org/docs

[C2] Chisel 官方安装：https://www.chisel-lang.org/docs/installation

[P1] Python subprocess：https://docs.python.org/3/library/subprocess.html

本地依据：题目/题目.png、会议记录/11.6.txt。工具名称推断与推荐的学习量、成果形式均是本资料的安排，不是导师逐字提出的验收标准。

01 数字电路与 RTL

本册目标：读懂电路，再写代码

电脑最终依靠电路处理信息。对入门学习而言，可以把导线上的低电平记为 0、高电平记为 1；这里的 0 和 1 表示逻辑状态，并非所有芯片都使用同一个实际电压。

电路里有两类基本构件：一类根据当前输入计算输出，另一类能够记住之前的值。前者组成组合电路，后者让系统拥有状态。

先建立三个概念

概念	通俗解释	本课题中的例子
信号	导线上随时间变化的值	“请求启动”“已授权”“要求停止”
逻辑	输入如何决定输出	只有请求且授权时才放行
状态	电路保存的历史信息	已收到停止命令，之后继续保持停止

RTL 是 Register Transfer Level，即寄存器传输级。它描述寄存器保存什么、在什么时刻更新，以及寄存器之间怎样计算数据。RTL 是一种描述层次，Verilog / SystemVerilog 是常用的描述语言。

软件与 RTL 的思考方式

软件通常让处理器按照指令执行任务；RTL 用代码描述要构成怎样的电路。两个独立的硬件模块可以持续同时工作，代码里的书写顺序不等于硬件逐条运行顺序。

例如两个连续赋值会描述两条并行逻辑路径：

```verilog
assign p = a & b;
assign q = c | d;
```

assign 表示持续连接逻辑；& 是按位与，| 是按位或，分号表示语句结束。只要相关输入改变，仿真器就会按规则更新输出；真实硬件还存在传播延迟。

> 必学：先把代码画成“输入 → 逻辑 → 输出”。本册所有电路都很小，不要求理解完整 CPU。

新增的“语法补充 V1-V7”在本册后半部分，带独立书签。第一次读代码时先查 V1-V3，遇到模块连接和仿真文件时查 V4-V5，V6-V7 用于动手巩固。示例中的片段不一定能单独编译，完整可运行文件另行标出。

1.1 二进制、位宽与数值

一个 bit 能表达两个状态

bit（位）就是一个 0 或 1。4 个 bit 组成的二进制数 1010，从右到左的位权是 1、2、4、8，所以十进制值为 8 + 0 + 2 + 0 = 10。8 个 bit 通常称为一个字节。

二进制	十进制	解释
0000	0	所有位都是 0
0011	3	2 + 1
1000	8	最高位为 1
1111	15	8 + 4 + 2 + 1

n 位无符号整数的范围是 0 到 2 的 n 次方减 1。“无符号”表示这里只解释非负数；有符号数还需要规定正负数的编码，本周选读即可。

代码里的写法

```VERILOG
wire [3:0] value;
assign value = 4'b1010;
```

wire 表示用于连接的信号；[3:0] 表示有编号 3、2、1、0 共四位；4'b1010 表示“4 位，二进制，数值 1010”。同一个值也能写成 4'd10，其中 d 表示十进制。

为什么必须关心位宽

4 位只能保存 0 到 15。如果把十进制 16 的二进制 10000 存入 4 位，只留下低四位 0000。因此本册的 4 位计数器从 15 再加 1 会回到 0。

这叫截断导致的模 16 行为。若做“最多允许创建 15 个实例”的安全计数器，就不能无条件让它回绕，否则计数可能重新变小。可以考虑饱和计数：达到上限后保持上限，并拒绝新的请求。

试一试

1. 0101 是多少？5 位无符号数最多是多少？
2. 4 位的 1111 + 0001，存回 4 位后是什么？
3. 如果实例计数只能增加，你会选回绕还是饱和？为什么？

1.2 逻辑门与真值表

把一句规则翻成布尔表达式

布尔量只有真和假两种值，常用 1 和 0 表示。与表示“同时满足”，或表示“至少一个满足”，非表示“取反”。

A	B	A 与 B	A 或 B	非 A
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

上表列出了所有输入组合及其输出，叫真值表。验证简单组合电路时，可以把所有组合遍历一遍。

例：请求且授权才放行

```verilog
assign allow = request && authorized;
```

request 是是否有请求，authorized 是请求是否获得授权。&& 是逻辑与，常用于判断条件。对单 bit 信号，它在本例中与按位与 & 的结果一致；对多位数据，二者含义不同，不要混用。

如果还要求“没有停止状态”，可写为：

```verilog
assign allow = request && authorized && !stopped;
```

! 是逻辑非。stopped=1 时，!stopped=0，所以无论另外两个条件是什么，allow 都为 0。

读代码的方法

先找输出 allow，再从右向左问三个条件：是否请求、是否授权、是否仍可运行。这样可以把表达式重新翻回自然语言。

试一试

1. request=1、authorized=0、stopped=0，allow 是多少？
2. request=1、authorized=1、stopped=1，allow 是多少？
3. “授权或请求就放行”为何不符合原规则？给一个反例。

1.3 组合电路：二选一选择器

输入决定现在的输出

选择器类似铁路道岔：sel=0 时把 a 接到 y，sel=1 时把 b 接到 y。它不记住上一次选择，不需要时钟。

```VERILOG
module mux2(input wire a, b, sel, output wire y);
  assign y = sel ? b : a;
endmodule
```

module 到 endmodule 定义一个电路模块。input 是输入端口，output 是输出端口。模块名 mux2 可理解为“二选一选择器”。三目运算符 条件 ? 真时取值 : 假时取值 直接描述选择关系。

a	b	sel	y
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0

另一种写法，理解即可

```VERILOG
always @(*) begin
  if (sel) y = b;
  else     y = a;
end
```

在传统 Verilog 中，这样赋值的 y 要声明成 reg；这里的 reg 是语言中的变量类型，并不能仅凭这个词断定产生了寄存器。SystemVerilog 常用 logic 与 always_comb 表达此类逻辑。

组合逻辑需要覆盖所有分支。如果只写 if (sel) y = b;，sel=0 时没有说明 y 应是多少，可能推断出“保存旧值”的锁存器。锁存器是另一类存储元件，本周先通过完整赋值避免无意引入。

与课题的联系

“允许访问 / 返回拒绝”也可以先抽象成选择逻辑，但真实访问控制还需要身份、权限来源和不可绕过的执行位置。一个选择器只展示规则如何变成电路。

练习：写出 sel=0 时选 b、sel=1 时选 a 的一行表达式。配套 mux2_tb.sv 会检查全部 8 种输入组合。

1.4 时钟、寄存器与复位

电路怎样记住过去

寄存器可以保存若干位的值。把时钟想成一个反复跳动的节拍信号。本册使用上升沿触发的寄存器：时钟从 0 变 1 的那个瞬间，寄存器根据输入更新；两次上升沿之间通常保持值。

```VERILOG
always @(posedge clk) begin
  if (reset) q <= 1'b0;
  else       q <= d;
end
```

posedge clk 表示时钟上升沿；q 是已保存的状态，d 是准备写入的新值；<= 是非阻塞赋值。这里 reset=1 时，在上升沿把 q 写成 0。

上升沿编号	reset	d	沿后 q
1	1	1	0
2	0	1	1
3	0	0	0
4	0	1	1

复位用于建立已知初始状态。本例是同步复位：需要等到上升沿才生效。异步复位能够在规定的复位信号变化时触发，不等待时钟；它的释放和时序也有工程要求。[C3]

为什么时序代码常用 <=

想象 a、b 是两个寄存器。在同一个时钟沿执行 a <= b; b <= a; 时，两者分别读取沿到来前的旧值，之后一起更新，因而可以交换值。<= 不表示“小于等于比较”，要结合语境区分。

选读：真实电路的时间要求

输入不能在采样瞬间随意抖动；寄存器有建立时间和保持时间。跨时钟域的信号可能需要同步器。仿真里理想的 0/1 跳变，不能直接保证物理电路在任何时刻都可靠。

练习：若 reset 在两个时钟沿之间短暂从 0 变 1 又回到 0，这个同步复位寄存器一定复位吗？

1.5 逐行读懂一个 4 位计数器

本册第一个有状态的完整模块

```VERILOG
module counter4(
  input wire clk, reset, enable,
  output reg [3:0] count
);
  always @(posedge clk) begin
    if (reset) count <= 4'd0;
    else if (enable) count <= count + 4'd1;
  end
endmodule
```

第一到四行定义接口：时钟、复位、计数使能 enable 是输入，4 位 count 是输出。第五行表示只有时钟上升沿才更新状态。

第六行优先处理复位。第七行只有在 enable=1 时加 1。当 reset=0 且 enable=0，代码不安排更新，寄存器保持旧值。这种“未赋值就保持”的写法在时钟触发的寄存器里符合预期；与组合逻辑缺少分支不同。

先预测，再运行

第几个有效上升沿	reset	enable	沿后 count
1	1	1	0
2	0	1	1
3	0	0	1
4	0	1	2

复位与 enable 同时为 1 时，先命中的 reset 分支决定结果，所以得到 0，不是 1。此处“优先级”直接由 if / else if 的结构决定。

最重要的一句解释

“count 的下一状态 = 复位时为 0；否则使能时为旧 count 加 1；否则为旧 count。”这就是从代码提炼状态转移规则。

练习：把计数器改成每次加 2，需要改哪里？4 位计数从 14 加 2 后是多少？为什么？

1.6 仿真与测试平台

怎样知道写出的行为符合预期

仿真器在计算机上模拟硬件描述的行为。测试平台 testbench 是专门给模块提供输入并检查输出的代码。被测模块简称 DUT，即 design under test。

配套文件中 counter4.sv 是设计；counter4_tb.sv 是测试平台。测试平台提供时钟，依次驱动 reset 和 enable，等待采样沿，然后比较 count 与预期值。

```verilog
reg clk = 0;
always #5 clk = ~clk;
```

这里 #5 表示仿真延时，~ 表示按位取反。结合测试文件开头的 timescale 1ns/1ps，时钟每 5 纳秒翻转一次，周期为 10 纳秒。此处 # 延时属于测试激励，不用来描述一般可综合的计数器硬件。

为什么在沿后稍等再检查

```verilog
@(negedge clk); reset = r; enable = e;
@(posedge clk); #1;
if (count !== expected) $fatal(1, "counter mismatch");
```

先在下降沿改变输入，让输入在下一上升沿前稳定；再等上升沿并留出仿真更新时间检查。这样避免测试程序与设计在同一仿真时刻争抢读写，产生不容易理解的结果。

!== 还会区分未知值 X 和高阻态 Z；未知的 count 不会被误当成正确。$fatal 表示测试失败并终止；$display 打印信息；$finish 正常结束仿真。

测试至少覆盖什么

- reset 与 enable 同时为 1：确认优先级。
- enable=0：确认保持旧值。
- enable=1：确认加 1。
- count 从 15 加 1：确认位宽引起的回绕。

PASS 表示这些被检查的情形通过。它不等于所有可能行为已经被证明正确，也不包含时序收敛与 FPGA 实测。

1.7 按操作系统准备仿真工具

macOS：终端 + Homebrew

先按 05 安装 Homebrew，再在 Mac 终端执行：

```bash
brew install python icarus-verilog
```

注意包名是 icarus-verilog，装好后命令名为 iverilog 与 vvp。Apple Silicon 与 Intel 使用自己的原生工具架构；不要手工复制 Windows 的可执行文件。[V2]

Windows：PowerShell 或 WSL

如果已有原生 Icarus，确认 `Get-Command iverilog,vvp` 能找到两个工具，即可运行本册代码。没有原生工具可进入 WSL Ubuntu，按下方 Ubuntu 步骤准备。Windows 与 WSL 的命令分别在各自终端输入。

Ubuntu / WSL：Ubuntu 终端

```bash
sudo apt update
sudo apt install python3 iverilog
```

准备好后选自己的入口

先进入“配套示例”目录，按系统任选一条：

```powershell
# Windows PowerShell
py -3 run_examples.py rtl
```

```bash
# macOS Terminal
bash macOS/run_examples.sh rtl
```

```bash
# Ubuntu / WSL Ubuntu
bash Ubuntu-WSL/run_examples.sh rtl
```

三个入口都检查选择器、计数器、手写停止门与已附带的 Chisel 生成模块。仿真输出统一写入 `.outputs/`。完整定位目录和安装步骤见 05。

1.7 运行并理解仿真结果

编译与运行，三个系统使用同样的参数

安装好 Icarus 并进入“配套示例”后，可以手工逐个运行：

```TEXT
iverilog -g2012 -s counter4_tb -o counter4.vvp counter4.sv counter4_tb.sv
vvp counter4.vvp
```

Windows 在 PowerShell、macOS 在终端、Ubuntu 在 Ubuntu 终端运行上面的两行。iverilog 负责编译设计与测试，vvp 执行仿真。[V1]

命令逐项解释

部分	含义
-g2012	按 SystemVerilog 2012 模式处理
-s counter4_tb	指定测试平台为顶层
-o counter4.vvp	指定编译结果文件
后面的两个 .sv	编译需要的设计文件和测试文件

预期能看到计数结果，最后一行为：

```TEXT
PASS counter4: reset/hold/increment/wrap
```

手工命令在当前目录生成 counter4.vcd 波形文件；入口脚本运行则生成在 .outputs/。有兼容的波形查看器即可打开，观察 clk、reset、enable、count。没有查看器也可先读文本输出与逐拍表。

选择器和安全门的运行方式相同，只需替换文件名和顶层名：

```BASH
iverilog -g2012 -s mux2_tb -o mux2.vvp mux2.sv mux2_tb.sv
vvp mux2.vvp
iverilog -g2012 -s safety_gate_tb -o gate.vvp safety_gate.sv safety_gate_tb.sv
vvp gate.vvp
```

常见错误：“找不到模块”时检查文件是否都传给编译器；“找不到命令”时检查安装；“找不到文件”时检查当前目录。先保存源文件，再重新编译，最后重新运行。

1.8 状态机与停止锁存

把“曾经收到停止命令”记下来

有限状态机 FSM 可以理解为“有限个状态，加上一组切换规则”。本例只有 RUN 与 STOP 两个状态，用一个 bit 保存：stopped=0 表示 RUN，stopped=1 表示 STOP。

```text
RUN --收到 kill_req 的采样沿--> STOP
STOP --无论 kill_req 是否撤回--> STOP
STOP --受信任的复位-----------> RUN
```

“锁存停止”在这里是功能描述，具体实现采用上升沿触发的寄存器，并没有使用电平敏感锁存器。

```verilog
always @(posedge clk) begin
  if (reset) stopped <= 1'b0;
  else if (kill_req) stopped <= 1'b1;
end
assign allow = request && authorized && !stopped;
```

一旦采样到 kill_req=1，stopped 变为 1。后续即使 kill_req 回到 0，寄存器也保持 1，因此继续阻止 allow。

采样沿	reset	request	authorized	kill_req	沿后 stopped	沿后 allow
1	1	0	0	0	0	0
2	0	1	1	0	0	1
3	0	1	1	1	1	0
4	0	1	1	0	1	0

注意本例在采样沿之后才进入 STOP。若 kill_req 在两次采样沿之间变化，不能宣称输出在任何物理时刻都立即归零。真正跨时钟域或紧急停止电路还需同步、可靠触发和完整的时序设计。

练习：如果把停止状态直接写成 `assign stopped = kill_req;`，当命令撤回时会怎样？这与当前需求有何区别？

1.9 从一个小门电路看到系统问题

停止标志只是控制链上的一环

安全门示例只控制 allow 这一条信号。它没有操作系统接口、Agent 身份识别、网络接口或存储控制，因此不会实际终止任何 Agent。

把它放入一条假想链路：

```text
受信任的停止命令
        |
        v
停止状态寄存器 --> 拒绝新请求 --> 系统接口执行停止
                                          |
                                          v
                                确认相关任务和副作用已收敛
```

需要继续问的五个问题

1. 谁能写停止位、谁能复位？恶意程序能否把它清掉？
2. allow=0 后，已经进入队列或已经发出的请求怎样处理？
3. CPU 停止后，GPU、DMA、远端服务是否仍会工作？DMA 可理解为设备自行搬运内存数据的能力。
4. 同一个 Agent 的多个进程如何关联到同一个控制域？
5. “停止完成”由谁确认，确认的范围和时间基准是什么？

本例中 reset 优先于 kill_req。两者同时为 1，会把 stopped 清零；若此时请求与授权都为 1，allow 可为 1。测试专门覆盖这个行为，是为了让你看到“优先级”会影响安全含义。生产设计必须明确复位权限与冲突策略。

小改动练习

需求变成“停止命令与复位同时出现时，停止优先”。应把判断顺序改为：

```verilog
if (kill_req) stopped <= 1'b1;
else if (reset) stopped <= 1'b0;
```

这也会改变启动阶段的行为，需要同步修改规范和测试。不要只改代码，不改预期。

> 与课题相关的思考：硬件可以提供更难被篡改的状态和检查点，但“检查点覆盖了哪些行为”仍需要系统层面设计。

1.10 自测与动手题

不看答案，独立完成

1. “把 q 改成 1”与“每个上升沿把 q 更新为 d”有什么不同？
2. 组合电路为什么要说明所有输入分支下的输出？
3. `wire [7:0] x` 有几位？若按无符号数解释，范围是多少？
4. 对本册计数器，依次输入 (reset, enable)：(1,0)、(0,1)、(0,1)、(0,0)、(1,1)，沿后计数分别是多少？
5. 两个寄存器初值 a=0、b=1，在同一个沿执行 `a <= b; b <= a;`，沿后是什么？
6. 停止门中 kill_req 从 1 回到 0，stopped 为什么不回到 0？
7. 停止门何时复位？reset 与 kill_req 同时为 1 时会怎样？
8. 测试输出 PASS，是否代表 FPGA 一定满足时序要求？说明理由。

动手 A：饱和计数器

把 4 位计数器改成达到 15 后保持 15。先写自然语言规则，再补代码：

```verilog
if (reset) count <= 4'd0;
else if (enable && ________) count <= count + 4'd1;
```

测试至少包括：14 加 1 到 15；15 再加仍为 15；禁用时保持；复位时回到 0。

动手 B：自己设计一个状态位

要求“连续运行时允许请求，一旦违规就保持禁用，只有受信任的恢复命令能重新启用”。先定义输入与状态，再写状态转移表。最后明确：违规和恢复同时发生时，哪一个优先？为什么？

本册完成标准

能解释计数器每一行的作用；能不运行程序写出逐拍表；能描述停止门的能力边界；能成功运行示例或保留一份完整的手工结果。

1.11 参考答案与复习卡

前面练习的答案

二进制：0101=5；5 位最大 31；4 位的 15+1 存回后为 0000。限制实例数量通常应避免计数回绕造成低估，可使用饱和计数并单独处理释放与计数一致性。

布尔逻辑：两个 allow 都是 0。“或”规则会使 request=1、authorized=0 也被放行，违反授权要求。反选选择器写为 `sel ? a : b`。

同步复位：若高电平没有覆盖任何采样上升沿，就不保证寄存器看到它。计数每次加 2 可把增量改成 `4'd2`；14+2 截断到 4 位后为 0。

停止状态若直接连 kill_req，则命令撤回后会解除停止，失去“记录已经停止”的能力。

自测答案

1. 前者未说明时序和保持条件；后者定义了采样时刻与寄存器行为。
2. 组合逻辑应仅由当前输入决定；不完整赋值可能引入保持旧值的存储行为。
3. 8 位，0 到 255。
4. 0、1、2、2、0。
5. a=1、b=0，两者按更新前的值计算。
6. 未命中更新分支时，寄存器保持旧值。
7. 在 reset=1 的上升沿复位；本例复位优先，stopped=0。
8. 不代表。功能仿真没有完整验证布线延迟、建立保持时间及 FPGA 实际实现。

饱和计数器空白可填 `count != 4'd15`。动手 B 没有唯一答案，但必须定义优先级、复位值、恢复权限，以及所有状态下的输出。

记住这六句

组合电路看当前输入；时序电路还看保存的状态。位宽决定表示范围。时钟沿决定采样时刻。复位建立已知状态。测试平台主动制造输入并检查输出。一个硬件标志位的正确性，不等于整个系统安全目标已经实现。

阅读来源

[V1] Icarus 入门：https://steveicarus.github.io/iverilog/usage/getting_started.html

[V2] macOS Icarus 安装：https://formulae.brew.sh/formula/icarus-verilog

[C3] Chisel 复位说明：https://www.chisel-lang.org/docs/explanations/reset

[C4] Chisel 时序电路：https://www.chisel-lang.org/docs/explanations/sequential-circuits

本册代码与练习为教学编写，完整测试文件位于“配套示例”。阅读来源用于核对工具和语言概念，没有照搬其完整教程。

语法补充 V1：读懂模块与信号

必学：先分清哪是名字，哪是语法

```VERILOG
// This is a single-line comment.
module gate8(
  input wire [7:0] data,
  input wire enable,
  output wire [7:0] out
);
  assign out = enable ? data : 8'd0;
endmodule
```

这是完整的组合模块。gate8、data、enable、out 都是作者起的名字；module、input、wire、assign 是语言规定的关键字。大小写敏感，enable 与 Enable 是两个名字。

模块端口放在圆括号内，各项用逗号分开，最后一项不加逗号；声明和赋值一般以分号结束。// 后到行末是注释，/* ... */ 是块注释，不参与电路功能。

wire、reg、logic 怎么区分

写法	本周怎样理解	注意什么
wire	连接信号的网络	常接 assign 或子模块输出
reg	传统 Verilog 的过程赋值变量	在 always 中赋值，不必然是寄存器
logic	SystemVerilog 的常用四态数据类型	可描述组合或时序变量，仍需看驱动方式
integer	整数变量	测试中常用于循环下标

本资料的 .sv 文件启用 SystemVerilog，但保留 wire / reg 以便你对照原有经验。初学时遵循“一个信号由一处明确逻辑驱动”，不要让多个 always 随意写同一变量。

位宽与常量速查

[7:0] 是从编号 7 到 0，共 8 位；最高编号不等于位数。8'd12、8'h0C、8'b00001100 表示同一个 8 位无符号值。d、h、b 分别代表十进制、十六进制和二进制。

1'b0 是一个 bit 的 0；8'bxxxx_xxxx 表示仿真中的未知位，z 表示高阻态。X 不是可以随便当作 0 使用的“默认值”。初学示例尽量明确写出位宽和复位。

小练习：把 gate8 改成 4 位需要改哪些地方？答案：两个 [7:0] 改为 [3:0]，零常量改为 4'd0，模块名可自行改为 gate4。

语法补充 V2：运算、取位与拼接

必学：相似的符号，可能算出不同结果

写法	含义	本周常见用途
+、-	加、减	计算下一状态
==、!=	相等、不相等	判断状态
===、!==	含 X / Z 的严格比较	测试输出
<、<=、>、>=	大小比较	上限判断
&&、!	逻辑与、逻辑非	判断条件
&、~、^	按位与、非、异或	逐 bit 计算

逻辑或写 ||，按位或写 |；左移和右移写 <<、>>。逻辑运算判断是否为零，按位运算逐位计算；对多位数据，两者结果可能不同。

```verilog
// Assume a = 4'b1010 and b = 4'b0101.
assign bit_result = a & b;       // 4'b0000
assign logical_result = a && b; // 1'b1
assign low_nibble = data[3:0];
assign highest_bit = data[7];
assign joined = {a, b};         // 8'b10100101
assign repeated = {4{1'b1}};    // 4'b1111
```

花括号表示拼接，左边参数放高位。a、b 各 4 位，{a,b} 共 8 位；{4{1'b1}} 得到四个 1。

三目选择与括号

```verilog
assign out = enable ? data : 8'd0;
assign allow = request && authorized && !stopped;
```

三目表达式描述选择器。复杂条件用括号明确分组，例如 (a == b) && enable。

保留进位时要明确扩宽

若 a、b 各 8 位，本例使用 {1'b0,a} + {1'b0,b} 得到 9 位加法。240+60=300，8 位结果只保留 44，9 位结果可以保存 300。不要仅凭常量写法猜测表达式位宽。

语法补充 V3：赋值与条件分支

必学：先问是否需要保存状态

```VERILOG
reg [7:0] y;
always @(*) begin
  y = 8'd0;
  if (enable) begin
    y = a;
  end
end
```

这是模块内部的组合逻辑片段，假定 enable、a 已声明。@(*) 表示相关输入变化时重新求值；先给默认值，再按条件覆盖。enable=0 时输出也是确定的，不会无意保留旧值。

如果有多种选择，可用 case：

```VERILOG
always @(*) begin
  case (op)
    2'd0: y = a;
    2'd1: y = b;
    default: y = 8'd0;
  endcase
end
```

case ... endcase 按值选择，default 处理未列出的情况。begin ... end 将多条语句组成一个块；if 只控制下一条语句或紧随的整个块。

时序逻辑用时钟沿触发

```VERILOG
always @(posedge clk) begin
  if (reset) q <= 8'd0;
  else if (enable) q <= d;
end
```

此处 <= 是非阻塞赋值：在采样沿根据旧状态计算，再安排状态更新。未命中分支时寄存器保持旧值。对比 if (a <= b) 中的 <=，那里是“小于等于”比较。

入门习惯：组合过程用 =，时钟过程用 <=；不要对同一个变量混用两者。SystemVerilog 常用 always_comb 对应组合过程，always_ff @(posedge clk) 对应时钟过程，变量常声明为 logic。

课题联系：停止与恢复谁优先，由 if / else if 顺序决定。先写清规则，再写分支，不要靠看起来相似的代码猜行为。[V3]

语法补充 V4：实例、参数与循环

必学：模块定义一次，可以使用多次

```VERILOG
mux2 u_select(
  .a(left), .b(right),
  .sel(choose), .y(result)
);
```

这是已有 mux2 的实例化片段。mux2 是模块类型，u_select 是这一个实例的名字。.a(left) 表示“子模块端口 a 接当前模块信号 left”；点左侧是子模块端口名，括号里是本层信号。

配套仿真文件里的 dut 也是实例名，常表示被测设计。SystemVerilog 的 dut(.*) 表示按同名自动连接；初学先用显式端口连接读清楚，再理解缩写。

parameter：构建时决定电路规模

```VERILOG
module pass_data #(parameter WIDTH = 8)(
  input wire [WIDTH-1:0] in_data,
  output wire [WIDTH-1:0] out_data
);
  assign out_data = in_data;
endmodule
```

完整模块默认是 8 位。实例化时可用 pass_data #(.WIDTH(4)) u_pass(...) 改成 4 位，省略号处需补实际端口。参数在构建电路时确定，不是运行中每个时钟沿可随意改变的控制信号。[V4]

选读：generate 的循环生成

```VERILOG
genvar i;
generate
  for (i = 0; i < 4; i = i + 1) begin : g_copy
    assign out_bits[i] = in_bits[i];
  end
endgenerate
```

假定输入输出各 4 位。这是生成四条连接，不是让一根导线在四个时钟内依次工作。仿真文件中的普通 for 循环若带 #1，则可按仿真时间依次制造输入。

不要用“出现了 for”判断是否消耗多个周期；要看它位于生成结构、组合逻辑还是测试过程，以及是否有明确状态与时序控制。

语法补充 V5：读懂你的仿真文件

必学：testbench 就是你以前写的仿真文件

```verilog
`timescale 1ns/1ps
module clock_demo_tb;
  reg clk = 0;
  always #5 clk = ~clk;
  initial begin
    $display("simulation starts");
    repeat (3) @(posedge clk);
    $finish;
  end
endmodule
```

这个完整小例子只演示时钟与等待。反引号开头的 timescale 设置时间单位和精度，不是普通引号。每隔 5 ns 翻转时钟，因此周期 10 ns；repeat 等待三个上升沿，再结束。

你在配套文件中会看到的语法

写法	它在仿真中做什么	怎样读
initial begin ... end	该过程从仿真开始执行一次	初始化与测试步骤
always #5 ...	无限重复翻转时钟	测试时钟发生器
#1	等待一个时间单位	此处为 1 ns
@(posedge clk)	等到时钟上升沿	先等待，再执行后面语句
task ... endtask	定义可重复调用的测试步骤	配套 tick 是测试作者起的名字
$display	打印文本和数值	%0d 十进制，%b 二进制
$fatal、$finish	报错终止、正常结束	便于自动判断结果

$dumpfile 与 $dumpvars 记录 VCD 波形；它们不产生被测电路的功能。# 延时、打印和任务中的等待在这里用于测试，不应直接当成一般可综合硬件的设计方式。

配套 tick 为什么先等下降沿

它在下降沿改变输入，下一上升沿由设计采样，再稍后检查输出。这样避免测试过程与寄存器更新在同一时刻互相争抢。task 是为减少重复测试代码，不意味着产生一块名叫 tick 的硬件。

练习：时钟周期是 10 ns，执行 repeat(2) @(posedge clk) 表示等两个上升沿，还是把时钟周期改成 2 ns？答案：等两个上升沿。

语法补充 V6：综合运用的小模块

运行一次，把语法与波形连接起来

配套 syntax_basics.sv 同时展示组合计算与寄存器更新；下面是其主要逻辑，接口见完整文件：a、b 是 8 位，op 是 2 位，y 是 8 位，sum_full 是 9 位，count 是 4 位。

```VERILOG
assign sum_full = {1'b0, a} + {1'b0, b};
always @(*) begin
  y = 8'd0;
  case (op)
    2'd0: y = a + b;
    2'd1: y = a & b;
    2'd2: y = {a[3:0], b[3:0]};
    default: y = 8'd0;
  endcase
end
always @(posedge clk) begin
  if (reset) count <= 4'd0;
  else if (enable) count <= count + 4'd1;
end
```

设 a=8'hF0（240），b=8'h3C（60），先手算：

op	y 应是多少	原因
0	8'h2C，即 44	300 存到 8 位，只保留低 8 位
1	8'h30，即 48	按位与
2	8'h0C，即 12	a 的低四位放高半字节，b 的低四位放低半字节
3	0	命中 default

sum_full 始终为 300，不受 op 影响。count 只在采样沿更新；组合计算 y 不必等待时钟。

在已配置好的 VS Code 中运行

“终端 → 运行任务 → 语法：Verilog 综合示例”，或在 WSL 的示例目录运行：

```BASH
python3 run_examples.py rtl --example syntax_basics
```

完整仿真文件为 syntax_basics_tb.sv。预期 PASS，输出还说明检查了 262144 组组合输入及计数行为。波形写入 .outputs/syntax_basics.vcd。

语法补充 V7：自测、答案与速查

不看答案，先解释五处语法

1. wire [7:0] data 为什么是 8 位？data[3:0] 又是多少位？
2. a=4'b1010、b=4'b0101，a&b 和 a&&b 有何不同？
3. .enable(run) 中，哪一个名字属于子模块，哪一个属于当前层？
4. 给寄存器写 if (q <= 8'd10) q <= q + 8'd1;，两个 <= 分别是什么？
5. 在组合 always 中只写 if、不写 else 或默认值，可能出现什么？
6. 模块里有一个 for，就一定每个时钟只处理一个元素吗？

参考答案

1. 编号从 7 到 0 共 8 个；低四位取片得到 4 位。
2. 按位与得到 4'b0000；逻辑与因两个操作数均非零而得到 1'b1。
3. enable 是子模块端口，run 是本层信号。
4. 圆括号里是大小比较，赋值位置是非阻塞赋值。
5. 可能让输出保存旧值，推断无意的锁存器；应完整定义组合输出。
6. 不一定。生成循环或可展开的组合逻辑可以形成并行硬件；逐周期处理要明确设计状态和时钟更新。

写代码时的五步检查

先标注端口和位宽；再区分组合输出与寄存器；为组合逻辑定义完整输出；为状态明确复位和更新条件；最后写仿真步骤并核对预期值。

同一个错误不要只改报错行

例如 “unknown module” 要检查模块定义是否加入编译；“port not found” 要核对点左边的端口名；位宽告警要检查丢弃高位是否符合需求。修正后重新保存、编译、运行。

补充来源

[V3] AMD Verilog 语言与过程：https://docs.amd.com/r/en-US/ug901-vivado-synthesis/Verilog-Language-Support

[V4] AMD 参数说明：https://docs.amd.com/r/2025.1-English/ug901-vivado-synthesis/Verilog-Parameters

本补充使用基础 Verilog 与少量 SystemVerilog；运行采用 -g2012 模式。零散片段说明假定已有接口，完整源文件才是直接运行入口。

02 Chisel 与工具链

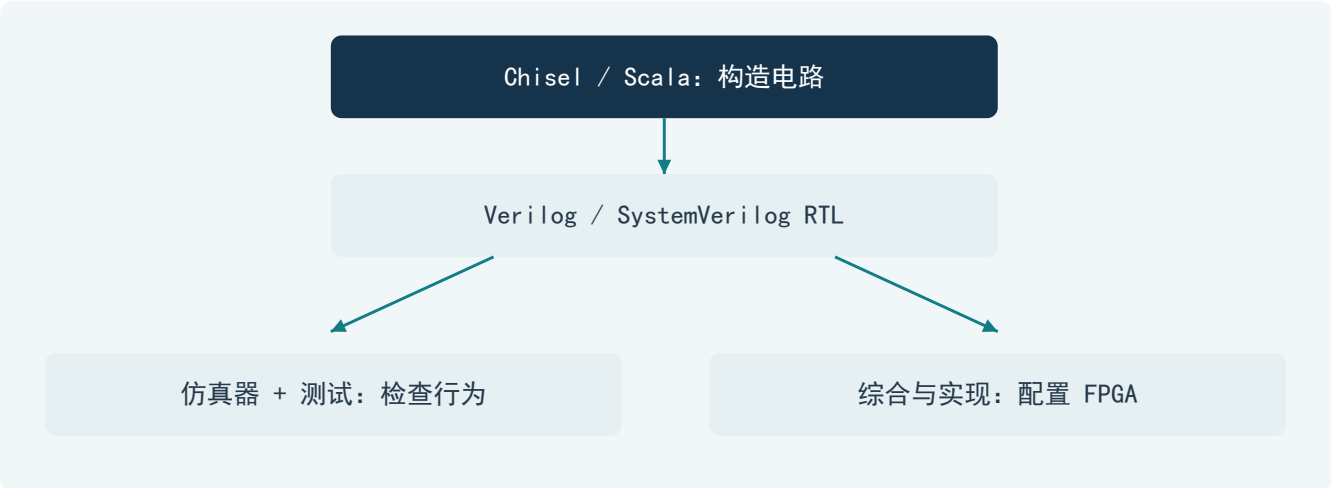
先把工具名字分清，再开始安装

会议前面写了 Trisim、Trisol，后面明确提到 Chisel 和 Chipyard。本资料按 Chisel 准备入门学习，这一名称推断仍待导师核实。

名称	是什么	你本周需要理解什么
RTL	一种硬件描述层次	状态如何在寄存器间流动
Verilog / SystemVerilog	硬件描述语言	描述接口、逻辑与寄存器
Chisel	基于 Scala 的硬件构造语言	用程序构造电路，生成 RTL
Scala	一种编程语言	读懂本册用到的少量语法
Icarus / Verilator	仿真相关工具	在计算机里检查 RTL 行为
Chipyard	集成多种生成器和工具的 SoC 研究框架	知道它在后续系统实验中的位置
FPGA	可配置逻辑硬件	后期把设计实现到真实器件上

SoC 是 System on Chip，片上系统，可包含处理器、内存接口、总线和外设。FPGA 可以实现这样的数字设计，但生成 RTL 之后仍需要综合、布局布线与板级验证。

最小工具链



Chisel 不是把任意 Scala 算法自动变成硬件。你要使用 Chisel 提供的构件，明确描述寄存器、连接与计算关系。[C1][C5]

常量、连接、比较等语法可查本册后半部分 C1-C7；先读小例子，再按需查阅。

2.1 只学够用的 Scala

六种写法就能读入门模块

写法	通俗解释	小例子
val	给一个值起名，名字不能重新指向别的值	val width = 4
class	定义一种类型；本册用来定义模块	class Gate extends Module
extends	基于已有类型扩展	extends Module
new	创建一个对象	new Bundle
{ }	把一组内容放在一起	模块、接口或条件块
import	使用库中的名字	import chisel3._

容易困惑的两种赋值

```scala
val limit = 4
val stopped = RegInit(false.B)
stopped := true.B
```

第一行创建一个 Scala 名字 limit，值为 4。第二行在构造电路时创建一个寄存器，并用 stopped 指向它。第三行使用 Chisel 的连接运算符 :=，描述这个硬件寄存器的更新输入。

所以 val stopped 并不表示“寄存器以后永远不能改变”。不可重新绑定的是 Scala 名字，硬件信号的值仍然可以随电路运行而变化。

Scala 数值与硬件数值

4 是 Scala 里的整数，可用于决定电路宽度。4.U 表示无符号硬件常量。4.W 表示位宽为 4。false.B 表示硬件布尔常量假。

```scala
val count = RegInit(0.U(4.W))
```

从里向外读：创建值为 0、宽度为 4 的无符号硬件常量，再创建以它为复位值的寄存器。

练习：val width = 8 与 val x = Wire(UInt(8.W)) 分别决定“构造电路的参数”和“电路中的信号”中的哪一个？

2.2 第一个 Chisel 组合模块

还是同一个二选一选择器

```scala
import chisel3._

class Mux2 extends Module {
  val io = IO(new Bundle {
    val a = Input(Bool())
    val b = Input(Bool())
    val sel = Input(Bool())
    val y = Output(Bool())
  })
  io.y := Mux(io.sel, io.b, io.a)
}
```

Module 是模块的基础类型。IO 定义模块边界上的端口，Bundle 把多个端口组织成一组，Bool() 表示 1 位布尔类型，Input / Output 说明方向。

io.y 表示访问 io 里的 y。Mux(条件, 真时输入, 假时输入) 对应 Verilog 的三目运算符。因此最后一行是 sel=1 时输出 b，否则输出 a。

把两种写法对起来

Chisel	Verilog 中相近的描述
Input(Bool())	input wire a
Output(Bool())	output wire y
io.y := Mux(io.sel, io.b, io.a)	assign y = sel ? b : a
UInt(4.W)	4 位无符号向量

这里只比较行为与接口的对应关系，生成的 RTL 名字和排版未必与手写代码一致。优化工具也可能化简逻辑。

“写了什么”与“生成了什么”

写 Chisel 时，你在执行一个构造电路的 Scala 程序；生成结果则是硬件描述。运行构造程序时执行的 println，只是在电脑上打印文字。电路上的运行时打印需要专门的仿真支持，不能混为一谈。

练习：假设 a=1、b=0、sel=1，y 是多少？这里需要时钟吗？

2.3 寄存器与条件更新

将 RTL 计数器翻译成 Chisel

```scala
import chisel3._

class Counter4 extends Module {
  val io = IO(new Bundle {
    val enable = Input(Bool())
    val count = Output(UInt(4.W))
  })
  val count = RegInit(0.U(4.W))
  when(io.enable) {
    count := count + 1.U
  }
  io.count := count
}
```

RegInit 表示带复位初值的寄存器。标准 Module 提供时钟与复位接口。when 用来描述硬件条件；没有命中更新条件时，这个寄存器保持原值。[C4]

本例将加法结果接回 4 位寄存器，超出位宽的高位无法保留，所以会出现与 01 中相同的模 16 计数行为。若需要显式保留进位，必须设计更宽的输出或寄存器。

Scala if 与 Chisel when 的区别

Scala 的 if 通常在构造电路时决定生成哪种结构，例如是否生成一个调试模块；Chisel 的 when 由电路运行时的硬件信号决定某条连接如何选择。

```scala
val width = 4
val count = RegInit(0.U(width.W))
when(io.enable) { count := count + 1.U }
```

width 在生成时决定寄存器有几位；enable 在电路运行时决定是否更新。这两个“时间”应当分开理解。

同步复位的理解

教学对应图按普通同步复位理解，具体生成的复位类型以模块及生成结果为准。Chisel 还支持异步与抽象复位类型；本周只需知道复位语义应当明确，不能只看变量名猜测。[C3]

练习：这个模块生成完成后，修改 Scala 里的 width 会立刻改变已经配置好的 FPGA 吗？需要经过哪些步骤？

2.4 实验准备：固定一组教学依赖

在已有环境里做最小实验

配套 chisel_project.scala 固定使用 Scala 2.13.16 与 Chisel 7.2.0，目的是让示例可复现，不表示这些是最新版本。编译器插件版本与 Chisel 版本配套，Scala 完整版本也要匹配。

```scala
//> using scala "2.13.16"
//> using dep "org.chipsalliance::chisel:7.2.0"
//> using plugin "org.chipsalliance:::chisel-plugin:7.2.0"
//> using options "-language:reflectiveCalls"
```

这些以 //> 开头的行是 Scala CLI 指令，告诉工具下载什么依赖。普通 // 则是代码注释。第一次运行需要网络、下载依赖及 Java 环境，后续通常可利用缓存。

Windows：PowerShell

WinGet 可用时，安装 Scala CLI：

```powershell
winget install virtuslab.scalacli
```

macOS：终端

先按 05 安装 Homebrew，再安装 Scala CLI：

```bash
brew install scala-cli
```

Ubuntu / WSL：Ubuntu 终端

按 [C6] 的 Linux 安装步骤准备 Scala CLI，已有安装可直接使用。三个系统安装后均以 scala-cli version 检查，首次依赖下载需要网络。

同一组教学依赖

本套入口指定 --jvm 21，让 Scala CLI 管理 Java 21，减少跟随系统默认 JDK 变化引起的问题。当前终端中 java -version 不成功，也不必然说明 Scala CLI 管理的 JDK 不可用。

> macOS 用原生架构的工具，Windows 后端不支持时可转 WSL。各系统完整步骤和报错处理见 05，不需要同时安装 SBT、Mill 和完整 Chipyard。[C2][C6]

2.5 生成自己的安全门 RTL

完整模块已经放在配套文件里

SafetyGate.scala 中最核心的部分如下；完整文件还包含 import、输入输出定义和生成入口。

```scala
val stopped = RegInit(false.B)
when(io.killReq) {
  stopped := true.B
}
io.stopped := stopped
io.allow := io.request && io.authorized && !stopped
```

这段代码与 01 的停止门有相同的主要状态行为：复位清除停止位，采样到停止请求后保持停止，allow 由请求、授权和停止位共同决定。

在配套目录运行：

```bash
scala-cli run chisel_project.scala SafetyGate.scala --main-class EmitSafetyGate --jvm 21
```

上面的命令在三系统安装好工具后均适用；也可用 05 的分系统入口并把末尾参数改成 chisel。 入口中的 ChiselStage.emitSystemVerilog 把电路生成成 SystemVerilog 字符串，再打印到终端。示例关闭了后端随机初始化附加代码，便于用 Icarus 检查；寄存器仍应按规范复位。结果应包含 clock、reset、请求与授权端口、stopped 寄存器及 allow 逻辑。

不要只找“一模一样的一行代码”

后端可能重命名内部信号或把条件更新化简。比较时检查：输入输出方向、寄存器初值、何时更新、未更新时是否保持、组合输出规则。这些比格式更重要。

必做推演

先复位，再输入请求与授权，再采样 killReq=1，最后撤回 killReq。把每一步 stopped 和 allow 的值写下来，并与 01 的逐拍表对照。

生成与仿真是两个步骤

成功打印 RTL 后，还需测试平台检查行为。配套提供 SafetyGate.generated.sv，是从本例实际生成结果中提取的主模块；适配文件把端口接到 01 的同一套测试上。示例说明给出运行命令与验证范围。

2.6 常见报错与小练习

按报错发生的位置排查

现象	常见原因	先做什么
找不到 scala-cli	未安装或 PATH 未更新	重新开终端，执行 version
dependency 下载失败	网络、仓库访问或版本错误	核对固定依赖，保留完整报错
提示 compiler plugin	插件缺失或版本不配套	同时使用 chisel_project.scala
把 Bool 用到 Scala if 出错	混淆构造时参数与硬件条件	对硬件条件使用 when
输出未连接	没有给所有输出路径定义值	从模块接口逐个检查
firtool / 平台错误	RTL 生成后端不可用或不匹配	查官方安装与平台支持

练习 A：授权门

给一个已有模块的输出接上规则：“有请求、授权有效、且没有暂停”。只写最后一行 Chisel 连接，端口名为 request、authorized、paused、allow。

练习 B：故意找区别

比较下面两段：

```scala
val x = Wire(Bool())
x := io.killReq
```

```scala
val x = RegInit(false.B)
when(io.killReq) { x := true.B }
```

第一段与第二段是否都有记忆？killReq 撤回后分别会怎样？

练习 C：停止优先级

已有同步复位模块默认复位会清除停止位。如果研究要求“停止优先于恢复”，你应先修改什么规范、再检查哪些输入组合？允许写自然语言，不要求本周实现复杂复位结构。

2.7 参考答案与工具链复习

本册练习答案

width=8 是构造电路的参数；Wire(UInt(8.W)) 创建 8 位硬件信号。二选一选择器在 a=1、b=0、sel=1 时输出 0，不需要时钟。

修改生成器参数后，要重新运行生成、综合与实现，并重新配置 FPGA，才能改变对应硬件结构。具体还需看工程构建流程。

练习 A：

```scala
io.allow := io.request && io.authorized && !io.paused
```

练习 B：Wire 直接跟随 killReq，不保存历史；寄存器版本在采样到 1 后记住它，命令撤回也不会自动解除。

练习 C：先定义停止与恢复的优先级、谁有权恢复、复位阶段是否允许输出；再检查同时到来的停止和恢复、复位期间请求、停止命令撤回、连续重复停止等情况。不要仅靠名字叫 reset 就默认它可信。

本周对 Chipyard 的学习深度

知道 Chipyard 将处理器生成器、外围模块与验证工具组织成系统平台即可。先能独立读小模块、生成 RTL、解释仿真，再研究将安全模块接入 SoC 的哪条路径。完整搭建 Chipyard 可留待平台与架构方向明确之后。[C5]

阅读来源

[C1] Chisel 官方介绍：https://www.chisel-lang.org/docs

[C2] Chisel 安装与后端说明：https://www.chisel-lang.org/docs/installation

[C3] 复位类型：https://www.chisel-lang.org/docs/explanations/reset

[C4] 时序电路：https://www.chisel-lang.org/docs/explanations/sequential-circuits

[C5] Chipyard 中的 Chisel：https://chipyard.readthedocs.io/en/stable/Tools/Chisel.html

[C6] Scala CLI 安装：https://scala-cli.virtuslab.org/install/

官方文档会更新。这里固定的教学版本用于示例复现；后续工程选型应核对导师平台与依赖兼容性。

制作验证：本例已在 Ubuntu / WSL 实际生成 RTL，并用 Icarus 通过与手写停止门相同的定向测试；未进行 FPGA 验证。详细环境见配套验证记录。

语法补充 C1：Scala 外壳与硬件类型

必学：同一份文件里有两种层次

```scala
import chisel3._
import chisel3.util._

class Pass8 extends Module {
  val io = IO(new Bundle {
    val in = Input(UInt(8.W))
    val out = Output(UInt(8.W))
  })
  io.out := io.in
}
```

这是完整模块定义，生成入口和依赖需要另行配置。import 引入库中名字，util 包含 Cat、switch 等工具。class 定义类型，extends Module 表示这是 Chisel 模块；花括号组织代码块。多数语句不必写分号。

从内向外读一个端口

UInt(8.W) 是“8 位无符号硬件类型”；Input 表示方向；Bundle 将端口打包；IO 将其定义为模块接口；val io = ... 给接口对象起名。访问里面的端口写 io.in 或 io.out。[C7]

常量与类型速查

写法	含义	与 Verilog 对照
Bool()	1 位布尔硬件类型	单 bit 条件信号
UInt(8.W)	8 位无符号硬件类型	8 位无符号向量
SInt(8.W)	8 位有符号硬件类型	signed 向量
12.U(8.W)	8 位无符号常量 12	8'd12
"h0C".U(8.W)	十六进制常量	8'h0C
"b1100".U(8.W)	二进制常量	8'b00001100
true.B、false.B	硬件布尔常量	1'b1、1'b0

val width = 8 中的 8 是 Scala 整数，常用于决定生成结构；8.U 则是硬件数值。// 与 /* ... */ 都可写注释；名字区分大小写。不要把 val 理解成“硬件值永远不变”。

语法补充 C2：连接、比较与位运算

必学：=、:=、=== 分别做什么

写法	实际含义	例子
val x = ...	Scala 建立名字与对象的绑定	val x = Wire(UInt(8.W))
x := y	描述硬件连接或更新输入	io.out := io.in
x === y	硬件相等比较，结果是 Bool	io.a === io.b
x =/= y	硬件不等比较	io.a =/= 0.U
x <= y	硬件小于等于比较	io.count <= 10.U

Chisel 的 <= 不承担 Verilog 非阻塞赋值的作用。寄存器用 := 描述更新关系；保存状态的能力来自 Reg / RegInit，而不是从某一种赋值符号自动推断。[C8]

```scala
val same = io.a === io.b
val allowed = io.request && !io.stopped
val bits = io.a & io.b
val inverse = ~io.a
val low = io.a(3, 0)
val first = io.a(0)
val joined = Cat(io.a(3, 0), io.b(3, 0))
```

此处是假定端口已声明的片段。&&、||、! 用于 Bool 的逻辑运算；&、|、^、~ 用于按位运算。切片用圆括号，单 bit 用 a(0)；Cat 的第一个参数放在高位。

两个容易看错的地方

在 Chisel 中用 === 比较硬件，不能照抄 Verilog 的 ==。而 Verilog 里的 === 常用于包含 X/Z 的严格仿真比较；不要把两门语言相同符号的所有语义画等号。

x.asUInt 是将位按无符号解释，x.asSInt 是按有符号解释，不会凭空增加精度。例如 8 位全 1 按 UInt 为 255，按二进制补码 SInt 为 -1。

复杂条件用括号，例如 (io.a === io.b) && io.enable。若看到类型错误，先看你把 Scala Boolean 与硬件 Bool 混在一起没有。

语法补充 C3：Wire、Reg 与分支

必学：让不同构件承担不同职责

构件	记忆能力	本周怎样用
Wire(UInt(8.W))	无	稍后连接的组合中间信号
WireDefault(0.U(8.W))	无	带默认连接的组合信号
Reg(UInt(8.W))	有	寄存器，但这里未规定复位值
RegInit(0.U(8.W))	有	复位时回到 0 的寄存器
RegNext(io.in)	有	把输入延迟一个采样周期

WireDefault 的“默认”是电路连接的默认选择，不是“开机只赋一次值”。组合信号需要完整连接；寄存器可在某些条件下不更新，保持原值。[C4]

```scala
val y = WireDefault(0.U(8.W))
when(io.enable) {
  y := io.a
}.elsewhen(io.fallback) {
  y := io.b
}.otherwise {
  y := 0.U
}
io.y := y
```

when / elsewhen / otherwise 依次描述条件选择，前面的条件优先。上例在 enable 与 fallback 同为真时选择 a；如果只是二选一，可用 io.y := Mux(io.sel, io.b, io.a)。[C10]

```scala
val count = RegInit(0.U(4.W))
when(io.enable) { count := count + 1.U }
```

这是状态更新片段，Module 提供相应时钟与复位。读右侧 count 时取当前状态，写左侧时描述寄存器下一状态的输入，不会形成“自己加自己直到无穷”的组合循环。

Scala if 与 Chisel when

if (width == 8) 由生成时的 Scala 参数决定要构造什么；when(io.enable) 由硬件运行时输入决定电路选择。不能把 Bool 直接放进 Scala if 充当 Boolean。构造时与电路运行时应始终分开。

语法补充 C4：模块实例、参数与数组

必学：把小电路接到大电路

```scala
val gate = Module(new SafetyGate)
gate.io.request := io.request
gate.io.authorized := io.authorized
gate.io.killReq := io.killReq
io.allow := gate.io.allow
```

假定这段放在父模块内，且父接口已声明。Module(new ...) 实例化已有模块；gate 是实例名。子模块输入在左边接父输入，父输出在左边接子输出。常规 Module 子模块还会继承相应的时钟和复位上下文。[C9]

Scala 构造参数决定结构

```scala
class PassN(width: Int) extends Module {
  require(width > 0)
  val io = IO(new Bundle {
    val in = Input(UInt(width.W))
    val out = Output(UInt(width.W))
  })
  io.out := io.in
}
```

实例化时 Module(new PassN(8)) 生成 8 位版本。require 在生成时检查参数。width 是 Scala Int，没有形成一个运行中能改变宽度的输入端口。

选读：Vec 与生成循环

```scala
val lanes = Wire(Vec(4, UInt(8.W)))
for (i <- 0 until 4) {
  lanes(i) := i.U(8.W)
}
io.out := lanes(io.index)
```

假定 io.index 为 2 位 UInt，io.out 为 8 位 UInt。Vec 包含四个 8 位元素；until 4 不包括 4。循环生成四项连接，动态索引则形成选择逻辑，不代表逐个时钟遍历。[C11]

Vec(4, UInt(8.W)) 是四项 8 位数据；UInt(4.W) 是一个 4 位数。涉及存储容量更大或多端口时再学习 Mem 等结构，本周先能区分这两种表达。

语法补充 C5：switch 与加法位宽

必学：直接对照 Verilog case

```scala
val y = WireDefault(0.U(8.W))
switch(io.op) {
  is(0.U) { y := io.a + io.b }
  is(1.U) { y := io.a & io.b }
  is(2.U) { y := Cat(io.a(3, 0), io.b(3, 0)) }
}
io.y := y
```

假定 a、b 和 y 为 8 位、op 为 2 位，已经 import chisel3.util._。没有匹配 is 时，y 使用之前给定的默认连接；这里是 0。可将它与 V6 中 case 的四种输出一一核对。

+ 与 +& 的区别

```scala
val truncated = io.a + io.b
val full = io.a +& io.b
```

当两个输入都是 8 位 UInt，第一种加法结果为 8 位，第二种扩展为 9 位。a=240、b=60 时，分别得到 44 和 300。即使把已经截断的结果接到 9 位输出，也不能找回丢掉的进位。[C12]

因此完整示例使用：

```scala
io.sumFull := io.a +& io.b
```

sumFull 的端口也声明为 9 位。端口、计算与接收寄存器三处位宽都要检查。初学时把端口和状态位宽写清楚，其他推断等熟练后再用。

常见错误的改法

容易写错	应先这样检查
用 a == b 比较硬件	改为 a === b，结果应为 Bool
用 Scala if 判断 io.enable	改为硬件 when 或 Mux
Wire 没有覆盖所有路径	提供默认连接或完整分支
只扩输出端口，不扩加法	需要进位时用 +& 并检查目标宽度
改 width 后期待旧 FPGA 自动变化	需要重新生成与实现

语法补充 C6：运行与 Verilog 同功能的电路

动手：先对照功能，再对照文本

配套 SyntaxBasics.scala 的完整接口与 V6 相同，主要逻辑如下。构建入口名是 EmitSyntaxBasics；不要误用前面安全门的入口名。

完整文件的 _root_.circt... 表示从根包查找生成工具，避免与 util 中同名包混淆；先知道用途，不必背这段导入路径。

```scala
val y = WireDefault(0.U(8.W))
switch(io.op) {
  is(0.U) { y := io.a + io.b }
  is(1.U) { y := io.a & io.b }
  is(2.U) { y := Cat(io.a(3, 0), io.b(3, 0)) }
}
io.y := y
io.sumFull := io.a +& io.b
val count = RegInit(0.U(4.W))
when(io.enable) { count := count + 1.U }
io.count := count
```

第一步：生成 RTL

VS Code“终端 → 运行任务 → 语法：Chisel 综合示例生成 RTL”。任务会选择本机已有的 Java 21；也可在配套目录直接执行：

```bash
scala-cli run chisel_project.scala SyntaxBasics.scala \
  --main-class EmitSyntaxBasics --server=false --jvm 21
```

此前 JDK 下载若卡住，仍可用 05 介绍的 --java-home 指向已有 Java 21。首次运行库和后端需要网络。

第二步：仿真已附带的生成模块

```bash
python3 run_examples.py rtl --example syntax_chisel
```

它使用 SyntaxBasics.generated.sv 和端口适配器，复用 V6 的同一仿真文件。这个命令检查仓库已有的生成结果，不自动把刚打印的多文件文本覆盖进去。

两种实现应满足相同输入输出与计数规则。生成工具可能化简内部逻辑，不能要求生成文本与手写 Verilog 完全一样。当前实验验证指定功能与输入集合，不等于完整系统安全证明。

语法补充 C7：两种语言对照与自测

先用已知 Verilog 读懂 Chisel

要表达什么	Verilog / SystemVerilog	Chisel
8 位输入	input wire [7:0] a	val a = Input(UInt(8.W))
8 位常量 12	8'd12	12.U(8.W)
组合选择	sel ? b : a	Mux(sel, b, a)
低四位	a[3:0]	a(3, 0)
拼接	{a,b}	Cat(a,b)
普通硬件相等判断	a == b	a === b
带复位寄存器	时钟 always、复位分支、非阻塞赋值	RegInit 与 when、:=
组合默认值	always 块开头赋默认值	WireDefault 或显式默认连接

这些是入门对应关系，不是逐字符翻译规则。特别是 Chisel := 接在 Wire 还是 Reg 上，决定它描述的是组合连接还是寄存器更新输入。

自测与答案

1. val 会阻止寄存器更新吗？不会，它固定 Scala 名字的绑定。
2. 已丢进位的结果接到宽输出能恢复吗？不能，应使用 +&。
3. a=8'hF0、b=8'h3C、op=2，y 是多少？8'h0C，取低四位拼接。

官方语法入口

[C7] 数据类型：https://www.chisel-lang.org/docs/explanations/data-types

[C8] 运算符：https://www.chisel-lang.org/docs/explanations/operators

[C9] 模块：https://www.chisel-lang.org/docs/explanations/modules

[C10] 选择器：https://www.chisel-lang.org/docs/explanations/muxes-and-input-selection

[C11] 数组：https://www.chisel-lang.org/docs/explanations/bundles-and-vecs

[C12] 位宽：https://www.chisel-lang.org/docs/explanations/width-inference

03 操作系统进程生命周期

本册主线：从一个文件，到一次运行，再到退出

程序通常是一组指令及相关数据，可以保存在磁盘文件里。进程是程序的一次运行实例，操作系统为它维护执行状态、内存映射、打开的资源和身份等信息。

同一个 Python 文件可以被运行两次，形成两个进程；同一个模型文件也可能被多个服务读取。因此“文件有几份”“运行了几个进程”“存在几个 Agent 实例”是三个不同问题。

电脑里谁负责什么

部件	可以怎样理解	与进程的关系
CPU	执行指令的计算部件	当前执行某个线程的指令
内存 RAM	存放运行中代码和数据的工作区	为进程映射所需内容
磁盘 / 文件系统	保存长期数据	保存程序、模型与日志
操作系统内核	管理资源与权限的核心程序	创建、调度和回收进程
应用程序	用户想完成的具体工作	通过系统接口请求资源

一个够用的类比

程序像菜谱，进程像照着菜谱做菜的一次活动，CPU 像厨师，内存像工作台。一个菜谱能被用多次；某次活动结束，菜谱文件仍然存在。

类比也有边界：进程由真实的内核结构和硬件状态支持，不是一个单独的文件夹。删掉程序文件不等于可靠结束已运行的进程，结束进程也不等于删除所有持久化数据。

本册约定

概念主要按 Linux / Unix 解释；macOS 也能学习本册的进程与 POSIX 基础，但 Linux 特定接口不能直接照搬。Windows 底层创建接口不同。配套 Python 使用跨平台接口，不要求先学 C。图中的 PID 数字仅作示意。

必学：进程是什么、如何创建和退出、有哪些状态、如何通信。cgroup、namespace 等机制放在选读层，先理解用途与边界。

3.1 内核、系统调用与权限

应用为什么不能随意控制机器

现代操作系统通常把普通应用放在受限制的执行环境中。应用要读取文件、启动进程、发送网络数据或申请内存，通常通过系统调用进入内核，由内核检查参数、身份和权限，再执行对应操作。

```text
应用程序：希望打开文件 / 创建任务 / 发网络数据
                              |
                              v
系统调用入口：切换到内核处理相应请求
                              |
                              v
内核：身份与权限检查、资源分配、调用驱动
                              |
                              v
硬件 / 文件系统 / 设备 / 网络协议栈
```

用户态与内核态可先理解为不同权限的执行状态。一次系统调用通常让当前任务进入内核处理，并不必然发生“从进程 A 切换到进程 B”。这是权限切换和任务切换的区别。

进程通常有哪些信息

- PID：进程编号。PPID：父进程编号。
- 身份与权限：例如所属用户、允许访问哪些资源。
- 地址空间：它可以访问的虚拟地址范围及映射。
- 打开的文件与连接：例如日志、管道、套接字。
- 执行状态：运行位置、寄存器、可运行或等待状态等。

PID 是管理编号，不是永久身份，进程退出后编号可能被再次分配。长期安全归属若只记录一个数字，需要考虑编号复用和状态变化的竞态。

连接到课题

如果应用能够绕过某个用户态检查直接调用系统接口，那个检查就不能独立承担强制保护。研究控制点时，应问“所有相关请求是否都必须经过这里”“恶意程序能否修改执行规则”。

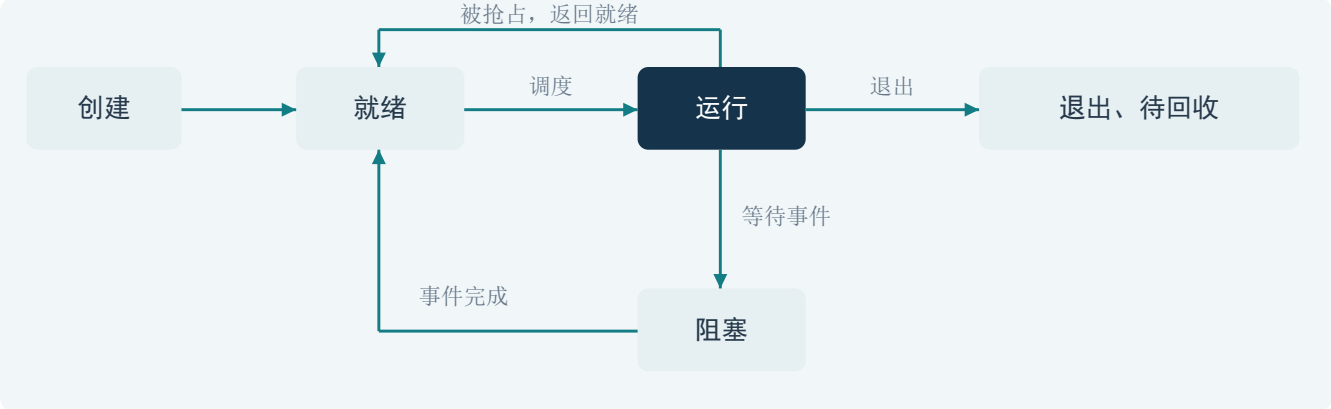
练习：某个 Agent 自己维护“是否允许启动工具”的变量，这个变量的可信程度取决于什么？

3.2 进程的状态与调度

有资格运行，不等于正在运行

状态	含义	生活类比
就绪	已具备运行条件，等待 CPU	排队等厨师
运行	此刻正在 CPU 上执行	厨师正在做这道菜
阻塞 / 等待	等待 I/O、时间或其他事件	等食材送到
已退出	执行已结束，可能仍待回收退出信息	工作结束，等登记结果

操作系统的调度器决定哪个可运行任务使用 CPU。时间片到期或更高优先级任务出现时，当前任务可能被切走；等待的事件完成后，任务通常先回到就绪，再等待调度。



上下文切换是保存当前任务的执行现场，再恢复另一个任务的现场。现场包括所需寄存器、下一条指令的位置及其他内核管理信息。一个核心上快速交替执行会呈现并发；多个核心还可能实现同时执行。

三个容易混淆的情况

进程调用 sleep 等待时间时，一般处于等待状态，而不是持续占满 CPU。进程“暂停”可以保留大量状态，之后可能恢复。进程“退出”则走资源清理与结果回收路径。

Linux 工具显示与教学状态图

ps 的 R 通常同时覆盖运行与可运行；S 是可中断睡眠；D 是不可中断睡眠，常与某些内核等待有关；Z 是僵尸状态。实际系统状态比入门四状态模型更细。

练习：某进程正在等待硬盘读完数据，它会一直占着 CPU 吗？事件完成后是否立刻在所有情况下运行？

3.3 创建、替换程序与等待

fork、exec、wait 各做一件不同的事

Linux / Unix 中，fork() 创建子进程。父子从 fork 后继续执行，但拿到不同返回值：父进程拿到子 PID，子进程拿到 0，失败时返回 -1。[01]

exec 系列接口把当前进程的程序映像替换成另一个程序。成功时通常不返回原程序，PID 保持不变。因此 exec 本身不能简单计为“新增了一个进程”。[02]

wait / waitpid 获取子进程状态；典型用法是父进程等待子进程退出，并让内核完成相应退出信息的回收。[03]

```text
父进程 P，运行程序 A
      |
      +--fork--> 子进程 C，起初也运行 A
      |                    |
      |                    +--exec--> C 改为运行程序 B
      |                                    |
      +--wait 等待-----------------------------+--exit
      |
      +--获得退出结果，继续运行
```

伪代码，只用于读流程

```text
child_pid = fork()
if 当前是子进程:
    exec(目标程序)
else:
    wait(child_pid)
```

父子谁先执行由调度决定，不应依赖“代码写在前面所以先输出”。实际程序还要处理失败和错误码。

Windows 与 Python 的对应关系

Windows 常用 CreateProcess 等接口创建程序实例，不能直接套用 fork 语义。Python 的 subprocess.Popen 把平台差异包装起来，适合先做跨平台实验；它在不同系统上的具体实现不保证都走同一条底层路径。[P1]

练习：一个 PID 从 Python 改为执行另一个可执行程序，PID 不变。这更接近 fork 还是 exec？

3.4 内存、虚拟地址与写时复制

两个进程里的“同一个地址”不一定是同一块内存

进程通常使用虚拟地址。CPU 的内存管理单元 MMU 根据操作系统配置的映射，把虚拟地址转换成物理内存位置，并检查相应访问权限。

```text
进程 A 的虚拟地址 0x1000 --> 映射 A --> 物理页 X
进程 B 的虚拟地址 0x1000 --> 映射 B --> 物理页 Y
```

因此不同进程可以使用相同的虚拟地址数字，却访问不同内容。页可以先理解为管理内存的一块固定大小单元，具体大小依平台和配置而异。

一个进程里常见的内容

代码区保存程序指令；栈保存函数调用中的部分临时数据；堆用于动态分配；映射区可能包含库、文件或共享内存。这里只建立位置概念，不要求记住地址布局。

fork 不一定立刻复制全部物理数据

Linux 采用写时复制 copy-on-write 优化普通 fork。父子在逻辑上拥有分开的地址空间，开始时一些物理页可共享；某一方写入需要私有的页时，再按规则复制相应页面。[01]

这解释了为何“创建了子进程”不能直接推断成“模型权重已物理复制了一整份”。大型模型还可能由共享推理服务提供，工具进程只向它发请求。

隔离也有例外与通道

共享内存允许进程在授权配置下访问共同区域；同一个文件、网络服务也能承载信息。拥有不同地址空间只是隔离的一部分，不能据此认定 Agent 无法协作。

选读：页表与保护

页表是描述映射和访问属性的数据结构。页表保护依赖内核正确配置与硬件执行；如果你的威胁模型允许内核本身恶意，就需要重新审视仅依赖内核的保护能否成立。

练习：两个进程都读取同一份只读模型文件，是否足以证明它们是同一个 Agent？说明理由。

3.5 进程、线程与 Agent 的关系

一个 Agent 可以跨越多个系统对象

线程是执行指令的单位；同一进程里的线程通常共享地址空间和许多资源，但各自保存线程执行现场。多个进程通常有分开的地址空间，也可能通过共享资源相互联系。

Agent 在本课题中可以先理解为：接收目标，借助模型选择行动，并通过工具持续执行的系统。harness 是组织提示、状态、工具、循环与错误处理的外围运行逻辑。

```TEXT
逻辑 Agent A
  |
  +--控制程序：保存任务、调用模型、决定下一步
  +--工具子进程：执行一次本地计算
  +--文件：任务状态、日志、可能的恢复材料
  +--网络连接：访问远端推理服务或外部工具

推理服务：可能同时服务 Agent A 与 Agent B
```

不能直接画等号的几组对象

- 一个模型权重文件不等于一个正在工作的 Agent。
- 一个进程可能只负责 Agent 的某项工具任务。
- 一个服务进程可能支持多个逻辑 Agent 会话。
- 一个 Agent 的状态可能保存在进程之外，供后续恢复。

怎样选一个工作定义

为实验定义“受控 Agent 域”：一个逻辑身份，加上被分配给它的运行任务、授权资源与允许的通信关系。后续再明确身份如何可信地绑定到进程、线程、设备队列与工具请求。

这是待设计的管理抽象。把几个 PID 写进列表，只是简单原型，还不能解决共享推理服务、并发创建、迁移和身份伪造。

练习：A 和 B 共用一个推理服务时，直接杀掉服务进程会有什么副作用？如果只按 PID 管理身份，会遗漏什么区分？

3.6 进程如何通信

地址空间分开，也可能交换信息

进程间通信简称 IPC。操作系统提供多种通信机制；应用还可通过共同文件或网络服务间接通信。对隔离课题，需要列出你准备控制的具体渠道。

渠道	信息怎样移动	本课题要检查什么
管道 pipe	一端写入字节，另一端读取	两端是否属于被允许通信的域
共享内存	共同访问一块内存	谁能建立映射、谁能读写
文件	一个写文件，另一个读取	目录权限、共享位置与继承句柄
本地套接字	同机端点交换消息	端点身份和访问权限
网络套接字	通过网络协议传输	目的地址、代理与外部中转
继承资源	子任务沿用已有资源引用	创建任务时继承了什么

文件描述符可以先理解为进程使用某个打开资源的编号；Windows 对应概念常称句柄。进程创建或程序替换时，某些资源可能被保留或继承，具体由接口与标志决定。

例：只拦网络还不够

假设 A 与 B 无法通过 TCP 发消息，但都能读写同一个目录。A 把文本写入文件，B 读取后就获得了信息。这是显式通道遗漏，无须先讨论复杂侧信道。

选读：隐蔽通道

即使显式 IPC 被控制，进程仍可能利用共享资源的时序变化传递少量信息。例如争用缓存影响另一方测到的时间。覆盖所有隐蔽通道通常是更强、更难的研究目标，应单独写入范围与假设。

练习：要验证“在已声明的显式渠道上隔离 A 与 B”，你会准备哪三类失败尝试？合法通信需要哪些对照？

3.7 退出、终止与回收

发出终止命令后，还需要观察结果

进程可以主动正常退出，也可以被外部机制终止。Linux 的 kill 系统调用用于发送信号，名称虽叫 kill，并非每种信号都表示终止。[04]

SIGTERM 常用于请求终止，程序可以处理、忽略或阻塞它；SIGKILL 不能被目标程序捕获、忽略或阻塞。但某些内核等待状态下，任务不保证在严格的短时间上界内立即消失。[04]

典型的父子回收路径

```text
子进程结束执行
      |
      v
内核保留少量退出信息：PID、状态等
      |
      v
父进程 wait / waitpid 获取结果
      |
      v
相应记录被回收
```

通常，已退出但还未被等待回收的子进程叫僵尸进程。它没有在继续执行用户代码，不能靠重复发终止信号让它“更死”；关键是回收路径。特定信号设置可使回收行为不同。[03]

父进程先退出会怎样

子进程不因父进程退出就普遍自动终止。Linux 中可能被其他回收者接管；Windows 也需要看显式设置的作业或管理机制。因此停止主程序不等于证明派生任务全部结束。

三个系统中的 Python 差异

Python terminate() 在 macOS / Ubuntu 等 POSIX 平台发送 SIGTERM，在 Windows 上调用 TerminateProcess；kill() 在 POSIX 上发送 SIGKILL，在 Windows 上是 terminate 的别名。不能把 Windows 的 terminate 理解成可处理的优雅退出请求。[P1]

练习：进程从工具列表消失、管道关闭、网络请求停止，这三种现象为什么不能互相完全替代？

3.8 实验一：创建一个子进程并等待

使用 Python，不需要先写 C

先运行配套 process_demo.py。下面是相同思路的简化版，便于逐行理解：

```python
import os
import subprocess
import sys

print("parent pid=", os.getpid(), flush=True)
code = "import os,time; print(os.getpid()); time.sleep(0.3)"
child = subprocess.Popen([sys.executable, "-u", "-c", code])
print("child pid=", child.pid, flush=True)
result = child.wait(timeout=5)
print("exit code=", result)
```

import 使用标准库；os.getpid() 获取当前 PID；sys.executable 是当前 Python 解释器路径。列表 [...] 把程序和每个参数分开传递。-c 后面是一小段要执行的 Python 程序，-u 让输出更及时。

Popen 启动子进程并返回管理对象。wait(timeout=5) 最多等待约 5 秒，否则报告超时；它本身不承诺超时就杀掉子进程。退出码 0 在本例表示子程序正常完成。

运行命令

```text
python3 process_demo.py
```

上面适用于 macOS 与 Ubuntu；Windows PowerShell 用 py -3 process_demo.py，已有 python 命令也可使用。预期输出含父子 PID、child done 及退出码 0。数字和部分输出顺序每次可能不同。

手工画图

把父 PID 写为 P、子 PID 写为 C，画出 P → C 的创建关系，再标记子进程打印、sleep 等待、退出、父进程 wait 返回四个事件。

练习：把子程序的等待改成 1 秒，父进程的等待是否仍会正常结束？timeout 是上限还是固定睡眠时长？

3.9 实验二：观察通信与终止

一条管道就能传递信息

运行 pipe_demo.py，预期输出为：

```
TEXT
parent received: hello from child
```

核心代码是：

```
PYTHON
result = subprocess.run(
    [sys.executable, "-c", "print('hello from child')"],
    capture_output=True, text=True, check=True, timeout=5,
)
print("parent received:", result.stdout.strip())
```

capture_output=True 收集标准输出与错误；text=True 按文本处理；check=True 在非零退出码时报告错误。父进程通过管道获得子进程的输出，这就是一次实际通信。

只终止自己创建的单个子进程

运行 terminate_demo.py。它启动一个打印 ready 后等待 10 秒的子程序；父进程读到 ready 后调用 terminate，随后 wait。POSIX 上等待超过 3 秒会再尝试 kill 并回收；Windows 下两种终止方法的底层语义不同，见 3.7。

输出类似 child ended; exit code = ...。具体数值与平台有关，不要求相同。这个实验只观察单个、已知的子进程，并没有实现进程树或 Agent 域的全量停止。

你应该记录什么

实验	观察内容	能说明什么
process_demo	父子 PID 与 wait 返回	创建、等待和退出结果
pipe_demo	父进程收到了子字符串	分开的进程可以通信
terminate_demo	已知子进程结束并被等待	终止请求与回收要配合

实验文件有完整清理逻辑。不要把示例改成无限创建任务；练习目的只是观察有限的生命周期事件。

3.10 选读：系统怎样组织隔离与管理

先知道机制的职责，不急着配置

机制	主要帮助解决什么	不能单独保证什么
用户与文件权限	限定身份能访问哪些对象	自动识别所有逻辑 Agent
namespace	为进程提供不同的资源视图	自动限制全部资源用量与通道
cgroup	分组管理任务与资源	单独提供所有信息流隔离
seccomp	限制可执行的系统调用	自动理解业务级合法行为
容器	组合多种系统机制包装应用	内核不可信时仍天然安全

namespace 可先理解为不同的“可见世界”，例如不同网络视图；cgroup 是管理分组；seccomp 是系统调用过滤。容器通常组合这些机制，并非另一台物理机器。

Linux cgroup v2 提供对组及其后代任务的管理接口，包括特定条件下的整组终止机制。它已考虑部分并发创建等问题，值得作为后续研究基线。[05]

对硬件研究的启发

先研究已有机制能在“内核可信、管理配置受保护”的条件下保证什么，再找出你的威胁模型增加了哪些要求。例如：身份标签需要不可伪造？控制状态需要独立于普通应用？哪些 CPU、内存或设备访问必须检查标签？

不能从“进程很多”直接推出“软件机制都做不到”。硬件辅助的研究价值，需要与已有软件机制在同样假设下比较，说明新增保护与代价。

只读观察命令，Ubuntu / WSL 可选

```bash
ps -o pid,ppid,stat,comm -p $$
cat /proc/self/cgroup
```

第一行查看当前 shell 的 PID、父 PID、状态与名字，$$ 是 shell 的自身编号；第二行查看执行 cat 的任务所属 cgroup。输出随系统配置变化，不要求与示意图相同。

本周无需修改系统 cgroup、权限或容器配置。能把“身份、分组、通信权限、停止范围”四个问题分开说明即可。

3.10 macOS 读者的对应关系

哪些能直接练，哪些先理解概念

内容	macOS 的学习方式	Linux / Ubuntu 的学习方式
创建、等待与退出	运行本套 Python 示例	运行同一套 Python 示例
fork / exec / wait 概念	阅读流程，理解 Unix 基础	阅读流程并结合 Linux 实现
terminate 与 kill	Python 采用 POSIX 信号语义	Python 采用 POSIX 信号语义
进程列表	ps 或“活动监视器”	ps、top 等工具
/proc、cgroup、seccomp	原生 Mac 不执行对应 Linux 命令	在支持的 Linux 配置中学习

macOS 的内核与 Linux 不同。某些进程状态字母、工具选项和底层实现细节也可能不同。因此本册的 Linux 实现细节不能全部当作 macOS 内核说明。

macOS：只读观察当前 shell

在 Mac 终端执行：

```bash
ps -o pid,ppid,state,comm -p $$
```

这可以看到当前 shell 的编号、父编号、状态和命令。再运行 Python 实验，把它打印的父子编号画成图。

Ubuntu / WSL：Linux 专项观察

```bash
ps -o pid,ppid,stat,comm -p $$
cat /proc/self/cgroup
```

第二行依赖 Linux 的 procfs 和 cgroup 接口。Mac 上出现“找不到文件”并不表示你没学会进程，而是所用系统没有相同接口。

本周无需先安装 Linux

用 macOS 原生环境完成三个 Python 实验、生命周期图和技术目标草案即可。之后若要实际配置 cgroup、namespace 或 seccomp，再选 Linux 虚拟机或导师提供的远程环境。

测试范围：当前制作环境已运行 Windows 与 Ubuntu，macOS 步骤依据官方文档核对，尚未在实体 Mac 实测。具体安装与入口见 05。

3.11 自测题

请用自己的话解释

1. 程序文件、模型权重、进程和 Agent 各是什么？能否一一对应？
2. 就绪与阻塞有什么区别？等待完成后为什么还可能要排队？
3. 系统调用与进程上下文切换是否总是一回事？
4. fork 新增了什么？exec 替换了什么？wait 获得并回收什么？
5. 写时复制为什么让“子进程数”和“模型物理副本数”不同？
6. 两个进程地址空间不同，为何仍能协作？举两个渠道。
7. SIGKILL 不能被应用忽略，是否就能保证任意时间上界内完成整个系统的停止？
8. 什么是僵尸进程？它还在运行用户代码吗？
9. 父进程退出后，所有子进程是否必然退出？
10. PID 为什么不适合作为永不变化的全局身份？

场景题：控制程序与共享推理服务

Agent A 的控制进程创建工具 T；A 与 B 共同调用推理服务 M；T 发起了一个网络请求 N。现在收到停止 A 的命令。

请画图并回答：哪些对象归 A 管？可以直接终止 M 吗？A 的控制进程退出后，你还要观察什么？如果 N 已经到达外部机器，本地停止能否撤销它已造成的影响？

场景题：限定你的隔离声明

你控制了共享文件、共享内存与本地套接字，但没有控制外部网络。请写一句准确的实验结论，再写一句过度夸大的结论，并说明两者差别。

完成标准

在纸上画完整生命周期；运行三个有限 Python 示例或推演输出；解释进程退出与 Agent 安全目标之间的差距。

3.12 参考答案与阅读来源

自测答案

1. 程序是指令与数据；权重是模型参数；进程是一次运行实例；Agent 是实现目标与行动循环的逻辑系统。它们没有普遍的一一对应关系。
2. 就绪任务等 CPU；阻塞任务等事件。事件完成后一般先成为可运行任务，再由调度器选择。
3. 不总是。当前任务可进入内核完成系统调用后继续运行，并未换成另一个任务。
4. fork 创建子进程；exec 替换当前程序映像；wait 获得子进程状态，并在典型退出路径中完成相关回收。
5. fork 可共享只读或写时复制物理页，且模型可能由共享服务提供。
6. 例如文件、管道、共享内存、网络连接。
7. 不能。还需考虑内核状态、并发任务、设备、已发请求与远端执行。
8. 已退出但保留少量信息等待回收的任务；不再执行用户代码。
9. 不必然；需明确整组生命周期管理机制。
10. PID 会复用，命名空间中的视图也可能不同。

场景 A：明确 A 的控制任务与 T 的归属；对共享 M 应按会话、请求或上下文控制，直接终止会影响 B；还要看 T、排队请求、连接与持久化恢复。已发生的外部影响通常不能靠本地停止自动撤销。

场景 B：可说“在所测配置下，A 与 B 无法通过列出的三个受控渠道传递测试消息”；不能说“它们绝对无法共谋”。后者超出了覆盖的渠道和行为证据。

阅读来源

[01] fork：https://man7.org/linux/man-pages/man2/fork.2.html

[02] execve：https://man7.org/linux/man-pages/man2/execve.2.html

[03] wait / waitpid：https://man7.org/linux/man-pages/man2/waitpid.2.html

[04] 信号：https://man7.org/linux/man-pages/man7/signal.7.html

[05] cgroup v2：https://docs.kernel.org/admin-guide/cgroup-v2.html

[06] namespace：https://man7.org/linux/man-pages/man7/namespaces.7.html

[07] seccomp：https://man7.org/linux/man-pages/man2/seccomp.2.html

[P1] Python subprocess：https://docs.python.org/3/library/subprocess.html

先阅读本册解释，需要进一步核实时再打开对应原文，不要求从头读完全部手册。

04 从高层目标到技术目标

本册目标：把愿景改写成能设计、能测试的问题

题目关注从处理器架构和软硬件协同出发约束 Agent 自我复制，并支持紧急强制终止。会议补充了多个 Agent 之间的隔离、防止协同作恶。

这些是研究方向，还没有完整说明“谁被控制、控制什么、什么情况下算成功”。本周最有价值的思考，是补上这些空缺，提出一版候选技术目标。

从熟悉的停车场理解

“让停车场安全”太宽泛；“只有获得授权的车牌才能抬杆，紧急停止后不接受新的抬杆请求”更具体。后者能找出身份、检查位置、状态、优先级和测试输入。

Agent 系统也需要这些要素，但“车辆”可能横跨控制程序、工具进程、推理请求、文件和网络行为。先定义这些对象怎样归属同一个受控域，再研究具体机制。

采用一个教学场景

本地有 Agent A 与 Agent B，各有控制任务和工具；它们调用一个共享推理服务 M。受信任管理者 C 设置启动和通信策略，并能够请求停止某个域。

```TEXT
管理者 C --> 启动许可 / 通信策略 / 停止命令
                    |
          +---------+---------+
          v                   v
       Agent A             Agent B
          |                   |
          +------> 服务 M <---+
```

这只是帮助讨论的模型。你最终使用哪种 Agent 框架、CPU 平台与隔离范围，需要在后续架构设计中确定。

> 本周输出一页技术目标草案即可。以下所有具体机制和数值都属于教学候选，不是已实现的毕设结果。

4.1 先写清楚假设

威胁模型：假定对方能做什么

威胁模型是研究的边界说明。可以先假设：Agent 能运行普通用户态代码、调用被授予的工具、尝试创建任务和发送消息；不能直接篡改受信任管理者、内核与受保护的硬件状态。

这只是一个起步版本。如果导师希望应对更强的对手，例如不信任操作系统内核，那么控制点、身份来源和验证方式都要重新审视。

要写清的字段	需要回答的问题	教学候选
保护对象	一个 PID、一个会话，还是一个任务域？	逻辑 Agent 域及其已登记任务
对手能力	能执行什么、修改什么、访问什么？	普通应用与授权工具接口
信任基础	哪些部分必须正确且不能被对手改动？	管理者、内核、受保护控制状态
系统边界	管本机、设备，还是外部机器？	第一阶段只覆盖本机明确的通道
触发来源	谁判断违规、谁发停止命令？	受信任管理者发出命令
暂未覆盖	哪些困难留作后续研究？	所有隐蔽通道、已发生的远端副作用

信任基础也叫 TCB，Trusted Computing Base。它不是越大越好，而是要明确依赖了哪些组件。硬件辅助通常希望减少某些依赖或增强强制执行能力，具体效果需要比较验证。

区分检测与强制执行

检测回答“是否发生了违规”；强制执行回答“即使对方不配合，怎样阻止行为”。本周可把停止触发设为管理者的人工命令，先研究执行路径，避免同时承担所有恶意行为检测问题。

关于 RSI 的处理

可以把更强的自主性、自我改写与复制能力作为威胁设想；实验可使用公开框架或有限的模拟任务。会议纪要关于未公开系统的说法不作为本资料已验证的事实。

练习：把“内核可信”改成“内核可能恶意”，你原先依赖的哪些保护还需要重新论证？

4.2 一套通用的转化方法

用七个字段写一条技术目标

字段	填写方式
对象	明确归属、资源与控制范围
违规行为	用可观察的事件描述，不依赖猜测意图
规则	规定什么允许、什么拒绝、何时切换
控制点	写出在哪里检查、怎样保证不能绕过
假设	列出信任的组件与不覆盖的渠道
指标	写出单位、计时起止点、成功与失败标准
实验	合法对照、违规尝试、边界与并发情况

从一句愿望到一句可测目标

愿望：“Agent 收到紧急命令后必须停止。”

候选技术表述：“在本机、内核与管理者可信的条件下，管理者接受对域 A 的停止命令后，控制器拒绝该域的新启动与发送请求，并等待已登记任务退出；分别记录拒绝开始时间和任务收敛时间，检查是否仍有受控活动。”

它仍需要进一步补充哪些发送接口、哪些任务与设备被纳入，以及需要达到的时间界限。但已经能据此画数据流、加日志、设计失败判据。

好指标的三项特征

可观察：能从受信任日志或检查点得到证据。可复现：别人用相同工作负载和配置能重复实验。与目标对应：测到的是保护效果，而不只是程序是否打印了“停止成功”。

一个明确的失败判据

若控制器已经记录域停止生效，但仍接受该域未经授权的新启动，这就是失败。只看主进程的退出码可能看不到它，因此观测点必须覆盖规则涉及的对象。

练习：把“防止共谋”改写成一句只涉及共享文件与本地套接字、能被实验反驳的目标。避免使用“绝对安全”之类无法界定范围的词。

4.3 目标一：可靠停止

把停止拆成阶段，分别计时

时间点	具体定义	可观察证据
t0	受信任控制器接受停止命令	命令接收日志
t1	域的新请求开始被拒绝	控制点的拒绝记录
t2	声明范围内的任务与在途操作收敛	任务状态及受控副作用检查

拒绝新请求的延迟是 t1-t0；收敛延迟是 t2-t0。这里没有包括“发现恶意行为所用时间”，因为触发来源在本实验里是人工命令。t2 必须先定义范围；若不包含远端请求，就不能用它宣称远端也已停止。

建议的候选指标

- 停止后受控新启动与新发送的接受次数应为 0。
- 声明范围内的活动任务数收敛到 0；明确观察窗口。
- 重复实验记录延迟分布、最大观测值、负载与失败次数。
- 同时运行的合法域 B 是否继续工作，作为误伤检查。

如果要写“最多若干毫秒”，应先有模型或实验依据，再说明平台、负载和假设。多次测试得到的最大观测值，并不自动构成所有情况下的严格上界。

从 RTL 示例到系统实验

停止门可以检查“采样到命令后，下一个状态为停止，allow 为 0”。系统还要处理域中任务、设备队列、打开连接和已发请求。时钟周期数与操作系统完成终止的毫秒数不能直接混成同一个指标。

四项必须覆盖的测试

正常任务中途停止；停止时恰有新任务创建；重复发送停止命令；停止与恢复命令同时到来。分别规定优先级与预期结果。

选读：测短时延可以用 Python `time.perf_counter()` 的时间差；若跨设备或机器计时，还需要考虑时钟基准与同步问题。[P2]

4.4 目标二：约束自我复制

先定义“复制成功”是什么

模型文件复制一份、创建一个普通工具进程、启动一个具备 Agent 行动能力的新实例、把任务状态移交到其他机器，可能属于不同事件。保护规则要围绕你的工作定义设计。

一个可讨论的候选是：“未经受信任管理者授权，不得在受控边界内激活一个具有独立行动能力的新 Agent 实例；所有合法派生任务应继承可追踪的域归属。”

为什么不直接禁止所有子进程

很多合法工具本身需要子进程。完全禁止可能破坏正常工作流。需要区分普通工具任务与新的 Agent 实例，或采用统一任务域管理，再为可激活能力、凭证和启动操作设置规则。

可能的路径	需要检查的问题
创建新任务	任务是否被绑定到正确域，是否超出配额？
在已有进程中激活新会话	没有新增 PID 时如何登记新实例？
保存状态后恢复	恢复时是否重新验证身份与授权？
调用外部执行服务	谁持有凭证，是否属于受控出口？

候选实验

授权启动一个实例应成功；未授权启动应拒绝；停止后即使旧授权存在也应按规范拒绝；多个并发请求不得绕过配额；合法工具任务仍能正常完成。

可记录“未授权激活成功数”“合法启动误拒次数”“域归属遗漏数”和性能开销。有限测试下未授权成功数为 0，是该测试集合的结果，不是对任意未知路径的数学证明。

本周草案要诚实保留的难点

如何识别一个通用程序已经成为新 Agent，可能很难。可以先限制实验框架，要求新会话与任务经过明确的激活接口，再讨论绕过该接口的可能性。这一假设必须写进结论。

练习：进程数没有增加，是否足以证明没有出现新的 Agent 实例？给一个场景。

4.5 目标三：限制跨 Agent 信息流

从“不能共谋”收敛到明确的通信规则

单靠观察输出，很难证明两个智能体绝对没有协同意图。可以先研究更具体的系统目标：在声明的通道集合上，未经授权的信息传递无法被接受。

例如规定 A 和 B 不能直接通信，但都可以访问推理服务 M 的独立会话：

发送方 → 接收方	A	B	M
A	本域内允许	拒绝	按独立会话授权
B	拒绝	本域内允许	按独立会话授权
M	仅返回 A 的响应	仅返回 B 的响应	服务内部规则

允许 A→M 和 B→M 不代表自动隔离。共享服务若把 A 的提示、缓存或文件暴露给 B，仍可能成为间接通道。需要明确服务本身是否可信、哪些状态可共享。

测试按渠道组织

- 共享文件：A 写入测试消息，B 在未授权目录或文件上读取应失败。
- 本地套接字：跨域端点连接或消息投递应被拒绝。
- 共享内存：未授权跨域映射应被拒绝。
- 网络：若纳入范围，就测试直接连接与已声明的中转路径。
- 合法对照：同域通信、授权服务返回应能工作。

结果应该怎样写

“在列出的接口、配置和测试工作负载下，未授权消息投递成功数为 0；授权通信正常完成。隐蔽通道和未纳入的外部服务仍需单独研究。”

性能代价也值得记录，例如通信延迟增加多少、正常任务吞吐下降多少。性能属于保护机制的代价评估，本课题的主要安全目标仍是阻止声明范围内的恶意行为。

练习：若所有本地 IPC 被隔离，但 A、B 都能访问同一个外部消息服务，你应如何限定结论？

4.6 软硬件协同从哪里入手

先画功能分工，不急着确定最终架构

```TEXT
受信任管理者
  | 配置身份、授权关系、停止 / 恢复
  v
受保护状态：域标识、权限、停止位、必要计数
  |                        |
  v                        v
软件接口检查           候选硬件检查点
启动 / IPC / I/O      请求归属 / 权限 / 停止状态
  |                        |
  +----------+----------+
             v
      执行结果、拒绝与完成确认
```

这是功能示意图。是否需要处理器指令扩展、内存标签、总线检查或设备配合，必须在威胁模型、平台与已有机制比较之后决定。

一个适合初学者思考的竞态

事件 1：创建任务的线程检查“域仍可运行”。事件 2：另一个线程把域设为停止。事件 3：创建线程沿用旧检查结果，提交新任务。

这叫检查与使用之间的竞态，也常简称 TOCTOU。不能只说“启动前检查一次就安全”。应明确生效点，以及检查、任务归属绑定、提交与停止之间如何排序或协调。

本周可以提出的最小原型

先做停止位与请求授权门的 RTL 模型；用软件逻辑模型表示域和通信；下一步再选择实际系统接口，把策略连接到真实执行路径。各阶段验证的问题不同，要分别报告。

比较对象

后续研究可比较“单任务停止”“已有组管理机制”“加入所提硬件辅助机制”，保持工作负载和信任假设可比。需要说明新增机制具体补足了什么保证，以及代价是什么。

练习：如果硬件只看一个由恶意应用自行填写的域编号，它能可靠知道请求属于谁吗？还缺什么？

4.7 动手：运行一个逻辑策略模型

用最少代码验证规则是否自洽

配套 policy_model.py 用一个集合记录活动实例、一个集合记录允许的通信边，以及一个 stopped 标志记录停止状态。集合是保存不重复成员的数据结构。

```python
domain = Domain()
assert not domain.start("B")
assert domain.start("B", authorized=True)
assert not domain.send("A", "B")
domain.edges.add(("A", "B"))
assert domain.send("A", "B")
assert not domain.send("B", "A")
domain.stop()
assert not domain.active
assert not domain.start("C", authorized=True)
assert not domain.send("A", "B")
```

运行命令：

```text
python3 policy_model.py
```

此命令用于 macOS / Ubuntu；Windows PowerShell 用 py -3 policy_model.py。也可运行 05 中自己系统的 Python 入口。 预期输出：PASS policy_model: 8 checks。assert 是断言，条件不成立时测试会失败；not 表示取反。start 返回逻辑上是否接受启动，send 返回是否允许发送，均不创建真实进程或网络消息。

你能从中学到什么

没有授权时启动被拒绝；授权边是有方向的，允许 A→B 不等于允许 B→A；停止后模型拒绝新启动与通信。

你不能据此证明什么

模型清空活动集合，不会终止操作系统中的任务。任意程序也能绕开这个 Python 对象直接操作系统。因此这只是规则演示，不能被描述成已实现的沙箱、硬件保护或 Agent 停止系统。

修改练习

不运行 stop，先添加 B→A 的授权边，再预测反向 send 的返回值。然后思考：如果要增加“最多两个活动实例”的规则，还需要处理释放、并发和重复名字中的哪些情况？

4.8 你可以填写的一页技术目标草案

建议带到下一次组会讨论

研究边界：第一阶段针对本机的受控 Agent 框架。管理者与内核暂定可信；实例激活与选定通信操作需要经过声明的控制接口。远端自主执行和所有隐蔽通道暂未纳入。

高层目标	候选技术目标	最小验证方式
硬停止	接受停止命令后拒绝域的新启动和发送，并跟踪已登记任务收敛	记录 t0/t1/t2；停止期间并发创建；检查残留及对其他域的影响
限制复制	未授权的新实例激活被拒绝，合法派生任务具有可追踪归属	授权 / 未授权对照；停止后启动；配额并发与恢复测试
多 Agent 隔离	选定的文件、共享内存与套接字渠道遵守跨域授权矩阵	未授权消息尝试；合法通信对照；共享服务状态检查

需要继续填写的空白

```text
受控 Agent 实体的工作定义：
所用 Agent 框架与运行平台：
域身份如何绑定到任务 / 会话：
纳入范围的具体接口与设备：
停止完成的判定与观察窗口：
预期时间指标及其依据：
软件基线能提供的保证：
拟由硬件增强的那一项保证：
当前最不确定的问题：
```

优先问导师的三个问题

1. 会议中的工具原名是否为 Chisel，预期使用哪套开发平台？
2. 第一阶段默认操作系统内核可信吗？管理边界是否先限定本机？
3. 毕设阶段希望优先实现哪一个目标，其他目标做到怎样的讨论或验证深度？

这些问题用于收敛后续研究范围。你本周可以先按明确写出的候选假设完成学习与草案，不需要等待全部架构细节确定。

4.9 综合自测与参考答案

先口头回答，再核对

1. 给停止目标加一个时间指标，至少要定义哪两个东西？
2. “复制权重文件”与“激活一个新 Agent”为什么不等价？
3. 为什么隔离目标要列出渠道，而不能只说“不同进程”？
4. 控制器打印“已停止”，是否足以验证所有任务停止？
5. 策略模型 8 条断言通过，能否证明真实操作系统不能被绕过？
6. 停止与创建并发时，为什么只在开始检查一次可能失败？
7. 第一阶段默认内核可信，后来增加硬件检查点，是否就自动不再信任内核？
8. 五天后的技术目标草案至少应包含哪些内容？

参考答案

1. 定义计时起止点与观察范围，还要说明平台、负载、成功失败判据。
2. 权重是数据，行动能力还依赖运行逻辑、状态、工具与授权；新会话也可能共享权重。
3. 进程可经文件、共享内存、套接字等传递信息，不同地址空间不能覆盖这些路径。
4. 不足。需要在可信观测点检查声明范围内的活动、在途操作和停止后的新请求。
5. 不能。模型验证规则，系统保护还要求真实接口上的强制执行与覆盖。
6. 检查后状态可能改变；提交时若沿用过期结论，就可能在停止后创建任务。
7. 不自动。需重新分析身份、配置、异常处理与设备路径还依赖哪些软件。
8. 对象、威胁与信任假设、规则、控制点、指标、实验，以及待导师明确的问题。

你应能说出的三句话

“我知道要保护哪个域、域里有哪些对象。”

“我能用事件和接口描述允许与拒绝的规则。”

“我能设计一个可能让这条规则失败的实验，并说明通过测试后结论能覆盖到哪里。”

阅读来源

[P2] Python 时间测量：https://docs.python.org/3/library/time.html

[05] 既有组管理机制：https://docs.kernel.org/admin-guide/cgroup-v2.html

本册的目标拆分、场景、策略模型与草案为教学设计，研究方向来自本地题目说明与 11.6 会议纪要。

05 按操作系统运行代码

只看自己的系统，跟着做一次

这册是前面四个主题的配套操作手册，不额外增加第六天。Windows、macOS、Ubuntu / WSL 的源代码共用，安装方式、终端语法和部分进程行为分开说明。

目录与运行入口

```text
学习资料 / 配套示例 /
  macOS / README.md、run_examples.sh
  Windows / README.md、run_examples.ps1
  Ubuntu-WSL / README.md、run_examples.sh
  run_examples.py        共用启动器
  *.py、*.sv、*.scala    共用教学源代码
```

先找到“配套示例”，在终端进入该目录，再运行对应入口。启动脚本只执行例子，不会替你安装软件或修改系统配置。

系统	运行 Python 练习	安装工具的地方
macOS	bash macOS/run_examples.sh python	Terminal 终端
Windows	py -3 run_examples.py python	PowerShell
Ubuntu / WSL	bash Ubuntu-WSL/run_examples.sh python	Ubuntu 终端

末尾的 python 是练习组名称，不是文件名。换成 rtl 运行四组仿真；换成 chisel 生成 RTL。先完成 Python，再继续安装硬件工具。

先获得文件

如果从 GitHub 下载，本仓库当前是私有仓库，需要相应访问权限。已登录网页可选择 Code → Download ZIP；也可以在配置好 GitHub 身份验证后克隆。

解压 ZIP 的目录可能叫 Graduation-Project-main，克隆目录通常叫 Graduation-Project。下面的路径只是例子，请按自己的位置替换。不会定位文件夹时，优先使用系统说明中的拖拽或地址栏方法。

macOS ① 安装与定位文件夹

适用于 Apple Silicon 和 Intel Mac

用 Spotlight 搜索“终端”或 Terminal 并打开。先安装命令行开发工具；系统提示已安装时即可继续：

```bash
xcode-select --install
```

接着打开 Homebrew 官网，按首页说明安装，并完成安装器给出的 Next steps。官网见 [M1]；已有 Homebrew 可跳过此步骤。

装齐本周需要的工具

```bash
brew --version
brew install python icarus-verilog scala-cli
python3 --version
iverilog -V
scala-cli version
```

Homebrew 中包名为 icarus-verilog；安装后运行命令叫 iverilog 和 vvp。只做 Python 实验时，可先仅安装 python。Scala CLI 会管理启动器要求的 Java 21。[M2][M3]

进入代码目录

假设仓库在下载文件夹中：

```bash
cd "$HOME/Downloads/Graduation-Project/学习资料/配套示例"
pwd
ls
```

$HOME 表示自己的主目录；pwd 显示当前目录，ls 列出文件。如果解压目录有 -main 后缀，相应修改路径。也可先输入 cd ，把 Finder 中的“配套示例”文件夹拖进终端，再按回车。

找不到 brew 时

重新开终端，并执行安装器给出的路径配置。Apple Silicon 默认路径通常为 /opt/homebrew，Intel 通常为 /usr/local；按实际安装路径选择，不要把两个架构的工具混着用。详细命令见 macOS/README.md。

> 本次没有实体 Mac，macOS 命令依据官方文档编写与核对；后面的实际测试记录将 Windows / Ubuntu 与 macOS 分开标注。

macOS ② 运行全部教学例子

第一步：Python 的进程实验

在“配套示例”目录运行：

```bash
bash macOS/run_examples.sh python
```

启动器依次运行创建与等待、管道通信、终止与回收、策略模型。若想逐个学习，使用：

```bash
python3 process_demo.py
python3 pipe_demo.py
python3 terminate_demo.py
python3 policy_model.py
```

第二步：RTL 仿真

```bash
bash macOS/run_examples.sh rtl
```

会检查选择器、计数器、手写停止门及已有 Chisel 生成模块。输出应有相应 PASS；计数器波形位于 `outputs/counter4.vcd`。文本结果与逐拍表足以完成基础练习，波形查看器可选。

第三步：Chisel 构造与生成

```bash
bash macOS/run_examples.sh chisel
```

首次会下载固定依赖与后端。输出包含 `module SafetyGate` 等 SystemVerilog；这一步是生成，不等于 FPGA 实现或形式证明。

手工命令也能跑

```bash
iverilog -g2012 -s counter4_tb -o counter4.vvp counter4.sv counter4_tb.sv
vvp counter4.vvp
```

```bash
scala-cli run chisel_project.scala SafetyGate.scala \
  --main-class EmitSafetyGate --server=false --jvm 21
```

反斜杠放在行末表示 Bash 命令续行；PowerShell 不能照搬这种续行方式。也可把整个 Chisel 命令合成一行。

Windows：按 PowerShell 路径运行

Python 先行，硬件工具按需准备

先按 Python 官网安装 Python 3，并检查 `py -3 --version`。已有 python 命令、没有 py 时，可把下文 `py -3` 换成 python。

在文件管理器进入“配套示例”，在地址栏输入 powershell 并回车。也可以用作者的路径示例定位，路径应按自己电脑修改：

```powershell
Set-Location -LiteralPath 'D:\Projects\毕设\学习资料\配套示例'
py -3 run_examples.py python
```

若想逐个学习，执行 `py -3 process_demo.py` 等文件。目录里也有 Windows/run_examples.ps1；若脚本被执行策略拦截，直接运行上面的 Python 命令，无需修改全局策略。

RTL：原生 Icarus 或 WSL

已有原生 Icarus 时，先检查再运行：

```powershell
Get-Command iverilog,vvp
py -3 run_examples.py rtl
```

没有原生工具，可进入 WSL Ubuntu，再读下一页。PowerShell 与 Ubuntu 终端的安装命令需要分开，不能把 sudo apt 直接粘贴到 PowerShell。

Chisel：Scala CLI

WinGet 可用时安装，随后重新打开终端并回到代码目录：

```powershell
winget install virtuslab.scalacli
```

```powershell
scala-cli version
py -3 run_examples.py chisel
```

若原生 Windows 的生成后端不支持或失败，可使用 WSL；此前的 Python 实验不依赖该后端。Windows 逐步说明见配套 Windows/README.md。[M3][C2]

Ubuntu / WSL：在 Ubuntu 终端运行

Windows 用户先确认已经进入 WSL Ubuntu

原生 Ubuntu 与 WSL Ubuntu 在此使用相同命令。Windows 用户可先在 PowerShell 输入 wsl，再确认自己进入了 Ubuntu 环境。

```bash
sudo apt update
sudo apt install python3 iverilog
```

定位代码

WSL 中 D 盘路径示例：

```bash
cd '/mnt/d/Projects/毕设/学习资料/配套示例'
```

原生 Ubuntu 若克隆在主目录：

```bash
cd "$HOME/Graduation-Project/学习资料/配套示例"
```

两条路径选一条，并按自己的位置修改。接着运行：

```bash
bash Ubuntu-WSL/run_examples.sh python
bash Ubuntu-WSL/run_examples.sh rtl
```

Chisel 工具

按 Scala CLI 官方 Linux 安装步骤准备工具，已有安装可跳过。安装完成后执行安装器提示的 PATH 配置，重新打开终端，再回到代码目录。[M3]

```bash
scala-cli version
bash Ubuntu-WSL/run_examples.sh chisel
```

启动器使用 Java 21；首次运行需要下载依赖与后端。命令参数固定了教学依赖，但工具安装方式会随官方发布变化。

Linux 的进程列表与 cgroup 观察命令见第 03 册；这些专项接口需要在 Linux 环境中使用。

三系统结果怎样对照

观察行为，不要求每台机器打印相同编号

实验	应看到的共同现象	可能不同的部分
process_demo	父子创建关系、子程序退出、wait 返回	PID、部分输出顺序
pipe_demo	收到 hello from child	一般无需比较编号
terminate_demo	已知子进程结束并被等待	终止机制与退出码
policy_model	PASS，8 条断言通过	平台不改变其逻辑规则
RTL	所列定向测试通过	工具版本提示与日志格式

POSIX 平台上 Python terminate 发送 SIGTERM，kill 发送 SIGKILL。Windows 中 terminate 使用 TerminateProcess，kill 是它的别名。macOS / Ubuntu 的退出码可能为 -15；Windows 本次环境观察到 1。重点是理解其对应行为。[P1]

Mac 可以观察什么

```bash
ps -o pid,ppid,state,comm -p $$
```

也可打开“活动监视器”。这些帮助观察进程，但 Linux 的 cgroup、namespace 和 seccomp 属于另一套系统接口；需要实际操作时，再使用 Linux 虚拟机或远程主机。

RTL 入口第四项在检查谁

它检查仓库里已经提供的 SafetyGate.generated.sv。chisel 入口会打印本次生成结果，不会自动覆盖这个文件。重新比较时，应先按生成文件分隔标记提取主模块，再使用适配器和测试平台。

实际验证范围

此前的 Python 已在 Windows 与 Ubuntu 运行，RTL 与 Chisel 已在 Ubuntu / WSL 验证。本次还会检查新增启动器与脚本路径；macOS 尚无实体设备运行记录，不能把 Bash 在 Linux 上通过等同于 Mac 实测。

常见错误：先定位发生在哪一层

同一个错误，不要反复盲试安装

现象	优先检查	下一步
找不到文件或脚本	pwd / Get-Location 是否在配套示例	重新定位，不要改源码
找不到 python3 / py	Python 是否安装、命令名是否正确	按自己系统入口检查
Missing iverilog / vvp	仿真工具是否在 PATH	macOS 用正确的 Homebrew 包名
找不到 brew	安装器 Next steps 是否完成	按实际架构配置路径
scala-cli 下载失败	网络、仓库可达性与报错 URL	保留完整错误，核对官方文档
firtool 架构错误	系统架构与工具是否一致	Mac 避免混用 Rosetta 与原生工具
Mac 找不到 /proc	使用了 Linux 特定步骤	改看三系统对应关系
Windows 不允许运行 ps1	PowerShell 执行策略	直接用 py -3 run_examples.py

依赖仓库有问题时

只有确认下载故障来自 Maven 仓库选择时，可在 macOS / Ubuntu 对单次命令指定中央仓库：

```bash
COURSIER_REPOSITORIES=https://repo.maven.apache.org/maven2 \
  bash macOS/run_examples.sh chisel
```

Ubuntu 将路径换成 Ubuntu-WSL/run_examples.sh；Windows 原生不要照搬这种变量前缀语法。网络恢复前，可先读已有 RTL 并做逐拍推演。

若卡在 JDK 索引下载，且已有 Java 21，可用 chisel 组的 `--java-home` 参数指向已有 JDK；具体路径与命令见自己系统的 README。库和后端首次使用仍需联网。

文件名和路径

路径有空格、中文时要保留引号。脚本根据自身位置寻找共享文件，所以整个“配套示例”文件夹应一起保留。不要只拷贝 macOS 或 Windows 子目录。

学习时间仍按五天安排。遇到环境问题先做可运行的 Python 与手工推演，并记录报错，避免把全部时间花在安装上。

检查清单与官方入口

在自己的系统打勾

- 我知道当前终端属于 Windows、macOS 还是 Ubuntu。
- 我能定位“配套示例”，并找到共享的源代码文件。
- 我运行了 Python 组，能解释父子进程与等待回收。
- 我运行了 RTL 组或完成逐拍推演，能解释复位与保持。
- 我知道 Chisel 生成与 RTL 仿真是两个阶段。
- 我知道哪些第 03 册内容属于 Linux 专项。
- 我记录了自己的系统、架构、工具版本和实际结果。

运行记录模板

```text
系统 / 版本 / 架构：
终端类型：
Python / Icarus / Scala CLI 版本：
运行的命令：
输出或报错：
我能解释的现象：
还需要核实的问题：
```

官方参考

[M1] Homebrew：https://brew.sh/

[M2] macOS Icarus：https://formulae.brew.sh/formula/icarus-verilog

[M3] Scala CLI：https://scala-cli.virtuslab.org/install/

[C2] Chisel：https://www.chisel-lang.org/docs/installation

[P1] Python subprocess：https://docs.python.org/3/library/subprocess.html

[M4] Homebrew 路径与安装：https://docs.brew.sh/Installation

更完整的分系统说明位于配套示例中的三个系统目录；验证范围与教学边界见“示例说明与验证记录”。